

Linguaggi di programmazione per cominciare:

FreeBASIC e Free Pascal (autore: Vittorio Albertoni)

Introduzione

Con il linguaggio di programmazione scriviamo istruzioni che un computer deve eseguire.

Per come è costruito il computer, affinché queste istruzioni siano da lui capite, dovrebbero essere costituite da sequenze organizzate di 0 e di 1: l'unico linguaggio che il computer capisce, infatti, è il linguaggio binario, nel caso specifico chiamato linguaggio macchina.

Dal momento che sarebbe lungo e difficoltoso produrre una serie di istruzioni in questo modo si è pensato di rappresentare il maggior numero possibile di istruzioni con parole, quelle che i padri del BASIC chiamavano istruzioni simboliche, che sottendono la rappresentazione binaria delle istruzioni stesse.

Così diventa possibile scrivere le istruzioni utilizzando termini facili da ricordare e che possono sottendere anche lunghe serie di 0 e 1, risparmiando fatica e tempo.

Parole e regole per scriverle e combinarle tra loro costituiscono un linguaggio di programmazione e il linguaggio di programmazione deve possedere uno strumento atto a convertire quanto scritto con le parole in linguaggio macchina.

Alcuni linguaggi effettuano questa conversione una tantum, raggruppando una volta per tutte le istruzioni in un programma binario avvalendosi di uno strumento software chiamato compilatore. Altri linguaggi effettuano la conversione ed eseguono il programma (in questo caso meglio denominato script) istruzione per istruzione, avvalendosi di uno strumento software chiamato interprete.

In genere i programmi compilati sono più impegnativi da scrivere, in quanto se ne vede l'effetto solo a compilazione avvenuta, ma sono di più veloce esecuzione una volta compilati.

I programmi (script) interpretati sono di più lenta esecuzione ma spesso consentono una interattività cammin facendo che ne facilita la scrittura.

Moltissimi sono i linguaggi che sono stati creati per programmare computer, ciascuno concepito per fare bene alcune cose lasciandone altre in subordine.

Vi sono linguaggi maggiormente adatti a produrre software di sistema, altri maggiormente adatti a fare un po' di tutto senza brillare significativamente da nessuna parte, altri ancora specializzati a fare solo calcoli, ecc.

In questo manuale presento due linguaggi, nelle versioni oggi distribuite come software libero, concepiti per chi è alle prime armi: FreeBASIC e Free Pascal.

Il primo destinato a principianti della programmazione, che più facile di così non si può.

Il secondo, sempre per principianti, ma più didattico, destinato ad educare alla così detta programmazione strutturata, quella che richiede di aver chiaro prima ancora di cominciare che cosa si vuole fare esattamente, in modo da evitare spreco di risorse.

Anche se, a proposito di risorse, con i computer che abbiamo a disposizione oggi i problemi cominciano molto più in là rispetto ai tempi in cui sono nate le prime versioni dei linguaggi qui contemplati.

Rammento che il mio primo vero personal computer, dopo il semigiocattolo Commodore VIC 20, fu un Philips TC100 con un disco fisso di 24 MB: praticamente oggi non potrebbe nemmeno contenere il file audio di una canzonetta e, allora, ospitava il sistema operativo e gli applicativi per le cose più ricorrenti, ivi compreso un compilatore Turbo Pascal.

Indice

I	FreeBASIC	4
1	Installazione	5
2	Come funziona: il primo programma	6
3	Tipi base	7
4	Variabili e costanti	9
5	Operatori	10
5.1	Operatori aritmetici	10
5.2	Operatori di confronto	10
5.3	Operatori logici	11
6	Subroutines e funzioni	11
7	Funzioni precostituite	12
7.1	per interfacciarsi con lo schermo	12
7.2	per interfacciarsi con la tastiera	12
7.3	per lavorare con i file	13
7.4	per la matematica	14
8	Interattività con l'utente	15
9	Strutture di controllo	15
9.1	Esecuzione condizionale	16
9.2	Ripetizione	17
10	Grafica	19
10.1	Creare una finestra grafica	19
10.2	Disegnare	21
10.3	Tracciare	25
10.4	Rispondere al mouse	27
10.5	Rispondere alla tastiera	27
10.6	Creare movimento	29
11	Estensioni	30
II	Free Pascal	31
1	Installazione	31
2	Come funziona	32
3	Basi del linguaggio	33
3.1	Tipi di dato	33
3.1.1	Numeri	33
3.1.2	Caratteri	34

3.1.3	Stringhe	34
3.1.4	Boolean	34
3.2	Costanti e Variabili	35
3.2.1	Costanti	35
3.2.2	Variabili	35
3.3	Operatori	35
3.3.1	Operatori aritmetici	36
3.3.2	Operatori relazionali	36
3.3.3	Operatori logici	36
3.4	Procedure e funzioni predefinite	36
3.4.1	Funzioni aritmetiche	36
3.4.2	Funzioni trigonometriche	37
3.4.3	Funzioni per manipolare stringhe	37
3.4.4	Generazione di numeri casuali	37
3.5	Istruzioni	37
3.5.1	Istruzioni di assegnazione	37
3.5.2	Istruzioni condizionali	38
3.5.3	Istruzioni di controllo del flusso	39
4	Programmazione	39
4.1	Procedure	40
4.2	Funzioni	41
4.3	Unit	41
4.4	Programma	42
5	Interattività con l'utente	42
5.1	Programmi console	43
5.1.1	Output	43
5.1.2	Input	43
5.1.3	Semplici programmi console di esempio	44
5.2	Programmi con GUI	45
5.2.1	Output	47
5.2.2	Input	47
5.2.3	Semplici programmi con GUI di esempio	48
6	Abbellimento della console	51
7	Lavorare con i file	53

Parte I

FreeBASIC

Il linguaggio BASIC nasce nel 1964 presso l'Università di Dartmouth, ad opera di John Kemeny e Thomas Kurtz, per facilitare l'accesso al mainframe GE-225 anche a utenti non particolarmente dotati di conoscenze informatiche: tant'è vero che il nome del linguaggio è l'acronimo di **B**eginner's **A**ll-purpose **S**ymbolic **I**nstruction **C**ode (codice di istruzione simbolica di uso generale per principiante).

Nel 1975, quando viene messo in vendita il primo computer da tavolo ad uso personale, il MITS Altair 8800, Bill Gates e Paul Allen creano il linguaggio Altair BASIC per programmarlo, derivandolo dal BASIC di Dartmouth. Il successo è tale che, nello stesso anno, i due danno vita alla Microsoft e l'Altair BASIC diviene Microsoft BASIC, proprietà intellettuale della Microsoft.

Anche i primi adattamenti sviluppati dalla Commodore (per VIC 20 e Commodore 64) e dalla Apple avvengono su licenza Microsoft.

Altra ondata di successo coincide con il lancio del PC IBM 5150 del 1981, che incorpora un dialetto del BASIC chiamato BASICA: subito dopo, nel 1983, partendo da BASICA la Microsoft sviluppa una nuova versione del BASIC, chiamata GW-BASIC, che decreta il predominio del BASIC Microsoft per la programmazione dei personal computer¹.

Al GW-BASIC fanno seguito rilasci di altre versioni, sempre da parte di Microsoft.

Nel 1985 esce QuickBASIC, che, per la prima volta nella storia del BASIC, incorpora un compilatore per eseguibili DOS. Tutte le versioni precedenti richiedono un interprete per essere eseguite.

Nel 1991 esce QBASIC, versione ridotta di QuickBASIC, senza compilatore ma, nel frattempo, viene rilasciata la prima versione di Visual BASIC, cavallo di battaglia della Microsoft fino alla versione 6 del 1998 ed è tuttora possibile programmare in Visual BASIC su Visual Studio per il nuovo ambiente .NET Framework.

FreeBASIC nasce come software libero nel 2004 ad opera di una equipe guidata da Andre Victor ed è distribuito con licenza libera GNU GPL.

Ha una sintassi molto simile a quella di QuickBASIC, con cui ha un alto grado di compatibilità.

E' un linguaggio compilato ed è utilizzabile sui sistemi Microsoft Windows, DOS (in modalità protetta), Linux e FreeBSD.

In questo manuale mi propongo di presentare le caratteristiche principali del linguaggio, utili per una sua conoscenza di base, per la produzione di programmi di non grande impegno e per avere la preparazione necessaria ad accedere senza fatica alla documentazione ufficiale in vista della realizzazione di cose più impegnative.

A questo proposito devo ricordare che il BASIC è un linguaggio vecchio e, anche nelle sue riedizioni più recenti, dimostra tutti gli anni che ha: per realizzare cose impegnative, oggi come oggi, non mi rivolgerei al BASIC, che è nato e rimane una cosa per «beginners».

Nel tempo è stato arricchito per fare cose che vadano oltre il due più due, FreeBASIC ha introdotto elementi per la programmazione a oggetti, esistono librerie di arricchimento, ma per le cose un tantino impegnative diventa anche molto complicato e non è detto che ci consenta di raggiungere l'obiettivo.

¹Il nome GW-BASIC fu scelto personalmente da Bill Gates e nessuno conosce l'origine del GW. C'è chi dice sia un omaggio a Greg Whitten, il dipendente Microsoft che più si dedicò al progetto. C'è chi dice voglia alludere alle iniziali dei cognomi Gates e Whitten. Altra interpretazione vuole che si tratti delle iniziali delle parole Gee Whiz (perbacco) con cui Bill Gates accolse favorevolmente il risultato del lavoro di progettazione: come a dire perbacco! che basic!, alludendo alla ricchezza del linguaggio.

1 Installazione

FreeBASIC si trova su <https://www.freebasic.net/>.

Cliccando sul pulsante



apriamo la pagina per il download su SourceForge, dove troviamo i file binari per Linux e per Windows.

Aprendo dalla home page la pagina DOCUMENTATION, troviamo il manuale ufficiale completo in vari formati.

Linux

Il file più aggiornato che attualmente (settembre 2026) possiamo scaricare è

`FreeBASIC-1.10.1-linux-x86_64.tar.gz`.

Se lavoriamo su Ubuntu possiamo trovare i file binari appositamente creati per Ubuntu 14.04, 16.04, 18.04, 22.04.

Decomprimiamo il file scaricato, ci posizioniamo nella directory in cui l'abbiamo decompresso con il terminale e digitiamo, come super-utenti (`sudo` o `su`, a seconda del sistema su cui lavoriamo), il comando `./install.sh -i`, con che installiamo il compilatore nella cartella di sistema `/usr/local`.

Purtroppo questa installazione non installa tutte le dipendenze di cui c'è bisogno per un perfetto funzionamento del compilatore.

Queste dipendenze sono:

per Debian/Ubuntu:

```
gcc
libncurses5-dev
libffi-dev
libgl1-mesa-dev
libx11-dev
libxext-dev libxrender-dev libxrandr-dev libxpm-dev
libtinfo5 libgpm-dev
```

per Fedora:

```
gcc
ncurses-devel
ncurses-compat-libs
libffi-devel
mesa-libGL-devel
libX11-devel libXext-devel libXrender-devel
libXrandr-devel libXpm-devel
```

per OpenSUSE:

```
gcc
ncurses-devel
libffi-devel
xorg-x11-devel
```

Dobbiamo pertanto verificare che questi file siano installati con il gestore dei programmi e installare i mancanti.

Windows

Il file più aggiornato che attualmente (settembre 2026) possiamo scaricare è:

FreeBASIC-1.10.1-win64.zip per sistemi a 64 bit,

FreeBASIC-1.10.1-win32.zip per sistemi a 32 bit.

Decomprimiamo il file .zip in C:\, dove comparirà così una directory FreeBASIC-1.08.1-winXX contenente il compilatore e inseriamo il relativo percorso nella variabile di sistema PATH, in modo da poter lanciare il compilatore da qualsiasi directory.

2 Come funziona: il primo programma

I programmi che produciamo con il linguaggio FreeBASIC sono eseguibili, cioè funzionano senza alcun interprete anche se sul computer non è installato il compilatore del linguaggio, purché siano stati compilati su un sistema operativo dello stesso tipo di quello del computer su cui lavoriamo: un programma compilato su Windows non gira su Linux e viceversa.

La compilazione è la traduzione in linguaggio macchina del programma che abbiamo scritto in FreeBASIC, un linguaggio che ha sintassi e regole particolari, che vedremo, ma è di tipo umano, in lingua inglese.

Le istruzioni che compongono il programma si possono scrivere con un qualsiasi editor di testo, salvando il file con estensione .bas.

Per la compilazione si apre il terminale (in Windows si chiama prompt dei comandi e si apre con il comando `cmd` o `command`), ci si posiziona nella directory dove è stato salvato il file e si invoca il compilatore con il comando `fbcc` seguito dal nome del file .bas: dopo pochi istanti, nella stessa directory troveremo l'eseguibile compilato, in Linux denominato come il file .bas senza estensione, in Windows denominato come il file .bas ma con estensione .exe.

Il più semplice programma, nella tradizione del Ciao mondo, può essere questo:

```
print "Ciao mondo"
```

che, con la compilazione, genera un eseguibile lanciando il quale vediamo comparire a terminale la scritta Ciao mondo.

Se lavoriamo in Windows e vogliamo lanciare l'eseguibile semplicemente cliccandoci sopra, dobbiamo aggiungere alla fine del listato del programma l'istruzione `sleep`, che mantiene aperto il terminale fino a quando premeremo un tasto qualsiasi: in caso contrario il terminale scompare immediatamente dopo il lancio del programma senza consentirci di leggere quanto vi è scritto.

Bene sapere che il compilatore FreeBASIC è case insensitive, cioè non distingue tra lettere maiuscole e lettere minuscole: l'istruzione `print` potrebbe anche essere scritta `Print`, `PRINT`, `pRinT` e il compilatore la riconoscerebbe in ogni caso.

Per inserire commenti nel listato del programma, cioè scritte che non debbano influire sulla compilazione ma servano solo per chiarire certi passaggi ad un lettore umano, possiamo usare i seguenti simboli:

' (apice), inserito sulla stessa riga dopo altre istruzioni definisce un commento fino alla fine della riga;

Rem, inserito all'inizio di una riga definisce un commento per tutta la riga;

tra i simboli '/' e '/' definiamo un commento su più righe o sulla stessa riga all'interno di una serie di istruzioni.

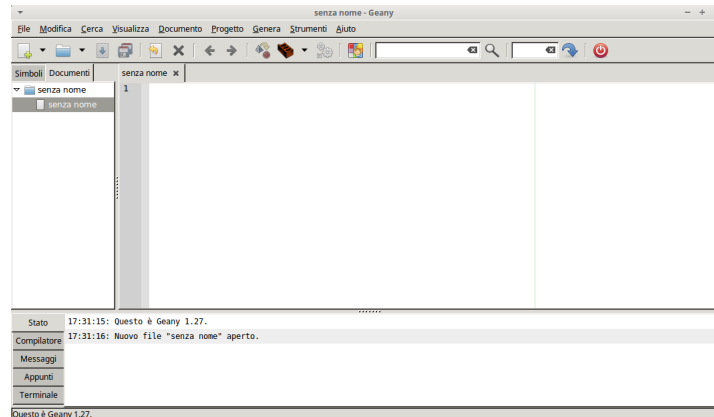
Per quanto riguarda la scrittura del listato del programma, che possiamo fare con un qualsiasi editor di testo (editor di testo e non word processor), è raccomandabile usare un editor pensato per la programmazione e che ci offra la possibilità di compilare il programma dallo stesso editor, semplicemente cliccando su un pulsante.

Un efficiente editor per programmare, di cui esistono versioni per Windows e per Linux, è Geany che troviamo all'indirizzo

<https://www.geany.org/download/releases/>

Chi usa Linux può installarlo facilmente attraverso il gestore dei programmi.

L'area di lavoro si presenta così



Passando il mouse sulle icone della barra degli strumenti sotto la barra dei menu ci viene spiegato a cosa serve ciascun strumento.

Con questo strumento possiamo scrivere il programma, compilarlo e provarlo senza muoverci dall'editor.

Per poter disporre della code completion dobbiamo salvare il file prima di cominciare a scrivere dandogli estensione .bas.

Chi usa Windows può trovare un altro buon editor all'indirizzo

<https://fbide.freebasic.net/download>,

che si installa con lo stesso meccanismo visto prima per l'installazione del compilatore e che ha il grande vantaggio di contenere nell'help il manuale ufficiale di FreeBASIC, purtroppo in lingua inglese.

3 Tipi base

Il linguaggio FreeBASIC è fortemente tipizzato e tutti i valori su cui agisce un programma devono avere un tipo. Il tipo assegnato ad un valore identifica la natura di quel valore, le operazioni che si possono effettuare su di esso e le modalità con le quali esso viene inserito nella memoria del computer.

Come tutti i linguaggi compilati, il linguaggio FreeBASIC è a tipizzazione statica, cioè richiede che il tipo di ciascun valore venga stabilito nel codice sorgente.

Qui vediamo i principali tipi previsti dal linguaggio.

Tipi numerici

I tipi che possiamo assegnare ad un valore numerico sono i seguenti.

Intero

Per i numeri interi, quelli che non hanno decimali, FreeBASIC prevede le seguenti alternative:

- . Byte e UByte con ampiezza di 8 bit,
- . Short e UShort con ampiezza di 16 bit,
- . Long e ULong con ampiezza di 32 bit
- . Integer e UInteger con ampiezza di 32 bit o 64 bit, a seconda del sistema,
- . LongInt e ULongInt con ampiezza di 64 bit.

L'iniziale U identifica tipi per numeri senza segno (unsigned), cioè solo positivi.

Per avere un'idea delle dimensioni massime del numero corrispondente ai vari tipi basta elevare 2 alla potenza corrispondente all'ampiezza in bit. Ciò fornisce la dimensione del tipo Unsigned. Per la dimensione del tipo Signed (senza la U iniziale) la dimensione del tipo Unsigned va dimezzata, metà con segno negativo e metà con segno positivo.

Esempio:

Per sapere la dimensione massima del tipo UShort a 16 bit eleviamo 2 alla sedicesima e otteniamo 65536.

Dal momento che in informatica si parte sempre da 0 la dimensione massima del tipo UShort è 65535 e il numero rappresentato potrà variare tra 0 a 65535.

Per sapere la dimensione massima del tipo Short, con segno, dimezziamo 65536 e otteniamo 32768 e il numero rappresentato potrà variare tra -32768 e 32767.

Il più grande intero concepibile con FreeBASIC è pertanto quello di tipo UInt

18.446.744.073.709.551.616

e se il risultato di un calcolo va oltre si genera stack overflow.

Decimale

I decimali sono i numeri reali, in informatica numeri a virgola mobile (floating point numbers) e sono quelli che contengono una parte decimale.

L'alternativa prevista da FreeBASIC è la seguente:

- . Single, con ampiezza di 32 bit (precisione singola),
- . Double, con ampiezza di 64 bit (precisione doppia).

Precisione singola significa avere 8 cifre significative, tra parte intera e parte decimale, precisione doppia significa averne 17, virgola compresa.

Lavorando con questi tipi non si va mai in stack overflow, fermo restando che i risultati dei calcoli hanno le precisioni indicate.

Tipo stringa

La stringa è una successione di caratteri e corrisponde all'identificativo string.

Il suo contenuto si crea scrivendo i caratteri all'interno di una coppia di doppi apici ".

Se ne può fissare la lunghezza facendo seguire all'identificativo il numero di caratteri preceduto da asterisco.

string *4 identifica una stringa di 4 caratteri.

I caratteri contenuti in una stringa possono essere cifre. In questo caso possiamo convertire la stringa in un vero e proprio tipo numerico con l'istruzione

val(<stringa>)

il numero che ne deriva è di tipo double.

Al contrario possiamo anche convertire un numero o una espressione numerica in una stringa con l'istruzione

str(<espressione_numerica>).

Tipo booleano

L'identificativo è Boolean ed ha ampiezza di 1 bit.

4 Variabili e costanti

Le variabili e le costanti sono delle locazioni di memoria, delle scatole, alle quali diamo un nome, destinate a contenere valori di un certo tipo.

Nel primo programma visto nel Capitolo 2 abbiamo usato l'istruzione `print` per visualizzare il saluto `Ciao mondo`.

In quel caso abbiamo indicato all'istruzione una stringa da visualizzare.

Nello stesso modo potremmo indicare un numero da stampare o anche un'espressione numerica, scritta utilizzando gli operatori che vedremo, per visualizzarne il risultato.

Il tutto utilizzando elementi volanti.

Se ci limitassimo a questo non andremmo lontano con i nostri programmi.

In un programma che debba fare qualche cosa in più di quattro conticini, che debba sviluppare algoritmi complessi nel corso dei quali siano da prendere decisioni su cosa fare in presenza di un valore piuttosto che di un altro è necessario tenere a disposizione i valori, poterli modificare e richiamare quando serve. A questo servono le variabili e le costanti.

Variabili

La variabile è una locazione di memoria destinata a contenere un valore modificabile nel corso del programma.

Essa va creata dichiarandone il nome e dichiarando il tipo di valore a lei destinato, eventualmente indicando anche un valore iniziale.

La sintassi per la dichiarazione di una variabile è:

```
dim <nome> as <tipo>
```

Per dichiarare più variabili dello stesso tipo possiamo scrivere

```
dim as <tipo> <nome>, <nome>, ....
```

Nel caso si voglia anche inizializzare la variabile, cioè assegnarle un valore iniziale:

```
dim <nome> as <tipo> = <valore>.
```

Se scriviamo `dim x as long` creiamo la variabile denominata `x` destinata a contenere un intero con ampiezza 32 bit con segno.

Se scriviamo `dim raggio as single = 7.56` creiamo la variabile `raggio` (per esempio di un cerchio) destinata a contenere un numero decimale con precisione singola e le assegniamo il valore iniziale 7,56 (attenzione alla virgola, che in FreeBASIC è un punto).

Se scriviamo `dim saluto as string = "Ciao"` creiamo la variabile `saluto` e le assegniamo il valore iniziale `Ciao`.

Se scriviamo `dim nome as string*10` creiamo la variabile `nome` destinata a contenere una stringa con un massimo di 10 caratteri.

Per i nomi possiamo utilizzare qualsiasi parola che non sia riservata al linguaggio.

Per il tipo possiamo indicare uno di quelli visti nel Capitolo 3.

Il valore può essere un numero, una espressione numerica, più o meno includente altre variabili, o una stringa (in tal caso sarà una successione di caratteri racchiusa tra apici).

Se quando dichiariamo una variabile vogliamo anche inizializzarla possiamo usare la seguente sintassi:

```
var <nome> = <valore>
```

In questo caso il tipo viene assegnato automaticamente in base al valore indicato.

Il programmino che abbiamo scritto nel Capitolo 2, se vi introduciamo l'uso di una variabile, diventa

```
dim saluto as string = "Ciao mondo"
print saluto
```

oppure, con la sintassi alternativa per le variabili inizializzate:

```
var saluto = "Ciao mondo"
print saluto
```

Come si vede, abbiamo innanzi tutto creato una variabile stringa che contiene la frase Ciao mondo e poi abbiamo detto a print di stampare quella variabile richiamandone il nome.

Il risultato è lo stesso che si ottiene con la scrittura del programma senza l'uso della variabile, ma ora abbiamo realizzato il vantaggio di avere nel programma una variabile utilizzabile in altre parti del programma semplicemente richiamandone il nome.

Costanti

La costante è praticamente una variabile il cui contenuto non può cambiare.

La sintassi per la dichiarazione di una costante è

```
const <nome> = <valore>
```

ove, per nome e valore vale quanto detto per le variabili nel precedente paragrafo.

Con questa sintassi il tipo viene automaticamente stabilito in base al valore indicato.

5 Operatori

Gli operatori collegano tra loro operandi di varia natura in espressioni che forniscono un risultato.

Quelli di uso più comune per i principianti sono i seguenti.

L'elenco completo degli operatori di FreeBASIC si trova nel manuale ufficiale in Language Documentation ▸ Operators List.

5.1 Operatori aritmetici

^ per l'elevamento a potenza

* per la moltiplicazione tra numeri

/ per la divisione tra numeri

\ per la divisione intera

mod per il modulo (resto della divisione intera)

+ per la somma tra numeri o stringhe

- per la sottrazione tra numeri

Gli operatori sono indicati in ordine di priorità di esecuzione delle relative operazioni.

Per modificare queste priorità nelle espressioni occorre usare le parentesi.

Esempio:

3 + 2 * 5 ritorna 13

(3 + 2) * 5 ritorna 25

5.2 Operatori di confronto

Servono per confrontare due valori e il risultato che restituiscono è un valore booleano.

Sono i seguenti.

= uguale,

<> non uguale,

< minore,

<= minore o uguale,

> maggiore,

>= maggiore o uguale.

Gli operandi assoggettati al confronto devono avere lo stesso tipo.

5.3 Operatori logici

Forniscono come risultato un valore booleano e sono i seguenti.

and,
or,
not.

6 Subroutines e funzioni

Il linguaggio FreeBASIC è un linguaggio procedurale, cioè il programma scritto in FreeBASIC altro non è che una serie di istruzioni che il computer deve eseguire una via l'altra.

Può accadere che, nel corso del programma, vi siano delle cose che vanno eseguite parecchie volte e in questi casi conviene raggruppare in blocchi le istruzioni per eseguirle in modo che, tutte le volte che serve, si possa semplicemente richiamare il blocco anziché scrivere ogni volta le relative istruzioni.

Questi blocchi sono praticamente delle sotto-procedure e si definiscono
. subroutines se eseguono compiti senza restituire valori,
. funzioni se restituiscono un valore.

Subroutines

La sintassi per costruire una subroutine è

```
sub <identificativo>  
    <istruzioni>  
end sub
```

dove <identificativo> è il nome che diamo alla subroutine per poterla eseguire quando serve e <istruzioni> sono le istruzioni da eseguire.

Quando e dove serve otteniamo l'esecuzione delle istruzioni semplicemente scrivendo il nome dell'identificativo assegnato.

La subroutine deve essere stata costruita prima di richiamarla.

Funzioni

La sintassi per costruire una funzione è

```
function <identificativo> (<argomenti>) as <tipo_dato_restituito>  
    <istruzioni>  
end function
```

dove <identificativo> è il nome che diamo alla funzione per poterla eseguire quando serve, <tipo_dato_restituito> è il tipo del dato che la funzione deve ritornare e <istruzioni> sono le operazioni da eseguire per ottenere quel dato.

Tra le istruzioni non può mancare l'istruzione return, destinata ad indicare il dato che la funzione deve ritornare.

Se per eseguire la funzione occorre indicare uno o più argomenti essi vanno indicati al posto di <argomenti> con nome e tipo, separati da virgola se sono più di uno, con la sintassi <nome> as <tipo>.

Quando e dove serve otteniamo il dato ritornato semplicemente scrivendo il nome dell'identificativo assegnato alla funzione e indicando tra parentesi tonde gli eventuali argomenti richiesti, separati da virgola se più di uno.

La funzione deve essere stata costruita prima di richiamarla.

Esempio:

Una funzione per calcolare l'area del triangolo può essere scritta così:

```
function area_triangolo (b as double, h as double) as double
return b * h / 2
end function
```

Per calcolare e scrivere l'area di un triangolo di base 5 e altezza 4 diamo l'istruzione

```
print area_triangolo(5, 4)
```

ed otteniamo scritto il risultato 10.

7 Funzioni precostituite

FreeBASIC contiene una miriade di funzioni già predisposte per fare determinate cose.

Qui non le esamineremo tutte ma solo quelle più ricorrenti per predisporre programmi alla portata di dilettanti.

Nel manuale ufficiale, sotto la voce Runtime Library Reference, troviamo tutte le funzioni suddivise per argomento. Cliccando sui loro nomi possiamo vedere a cosa servono e come si usano.

7.1 per interfacciarsi con lo schermo

FreeBASIC può interfacciarsi con l'utente semplicemente attraverso il terminale (quello che in Windows si chiama prompt dei comandi) o in modalità grafica, cioè attraverso una finestra sulla quale possiamo scrivere e disegnare, utilizzare colori, ecc.

Della grafica ci occuperemo in un capitolo a parte; per ora accontentiamoci del terminale.

L'unica funzione precostituita che ci interessa per lavorare su terminale è `print`, che già abbiamo utilizzato nel Capitolo 2 per il nostro primo programmino.

Questa istruzione scrive ciò che indichiamo dopo di essa. Se indichiamo una stringa (caratteri tra doppi apici) viene scritta tale e quale. Se indichiamo un numero o un'espressione matematica viene scritto il numero o il risultato dell'espressione matematica, automaticamente e internamente convertiti in stringa. Se indichiamo il nome di una variabile ne viene scritto il contenuto.

Possiamo anche indicare una lista di cose da scrivere sulla stessa riga, separando queste cose con un punto e virgola (;) o con una virgola (,).

Se usiamo il punto e virgola le cose vengono scritte una via l'altra, senza spazi intermedi.

Se usiamo la virgola le cose vengono scritte tabulate in spazi di 14 caratteri.

Se vogliamo risparmiare in battiture bene sapere che l'istruzione `print` può essere sostituita da un semplice punto interrogativo (?).

7.2 per interfacciarsi con la tastiera

La principale funzione di questa categoria è quella per acquisire dati dalla tastiera per inserirli in una variabile: `input`.

La sua sintassi è:

```
input <stringa_messaggio>, <variabile>
```

dove `<stringa_messaggio>` serve per formulare la richiesta di immissione del dato da parte dell'utente e `<variabile>` è il nome della variabile ove inserire il dato immesso dall'utente. La variabile deve ovviamente essere stata precedentemente dichiarata.

Altra funzione che può tornare utile per reagire alla pressione di un tasto da parte dell'utente è `inkey`, che ritorna una stringa rappresentante il carattere del tasto premuto.

7.3 per lavorare con i file

A volte può essere utile o necessario che il nostro programma salvi su disco determinate informazioni per archivarle e renderle disponibili per utilizzi successivi.

A questo scopo si crea un file in cui inserire o da cui estrarre informazioni.

Le funzioni di base per creare, alimentare e leggere il contenuto sono le seguenti:

`open` è la funzione che apre un file (se ancora non c'è lo crea) per lavorarci. Con essa si deve indicare il nome del file, il motivo per cui lo si apre e numerarlo. La sintassi della funzione è

```
open <nome_file> for <motivo> as #<numero>
```

dove

`<nome_file>` è una stringa contenente il nome del file con il percorso per raggiungerlo,

`<motivo>` è rappresentato da una delle seguenti tre parole:

output se nel file si devono indirizzare dati dal programma,

append se al file si devono aggiungere dati dal programma,

input se dal file si devono estrarre dati per il programma,

`<numero>` serve per fare riferimento al file per lavorarci;

`close` è la funzione che chiude il file e che consolida il lavoro fatto su di esso. La sintassi della funzione è

```
close #<numero>
```

dove `<numero>` è il numero che avevamo assegnato al file con l'istruzione `open`.

`print` è la funzione con la quale inseriamo dati nel file. La sintassi è:

```
print #<numero>, <dati>
```

dove

`<numero>` è il numero che avevamo assegnato al file con l'istruzione `open`,

`<dati>` è ciò che vogliamo inserire nel file, come stringa tra apici o numero.

`line input` è la funzione con la quale estraiamo dati dal file. La sintassi è:

```
line input #<numero>, <variabile_destinazione>
```

dove

`<numero>` è il numero che avevamo assegnato al file con l'istruzione `open`,

`<variabile_destinazione>` è una variabile, in cui inserire ciò che estraiamo dal file;

la variabile deve essere stata precedentemente dichiarata.

La funzione `line input` legge una sola riga del file; per leggere un intero file con più righe occorre inserirla in un ciclo (secondo uno dei metodi che vedremo in capitolo dedicato).

`eof` è la funzione che identifica la fine del file e restituisce `true` (1) alla fine del file. La sintassi è:

```
eof(<numero>)
```

dove `<numero>` è il numero che avevamo assegnato al file con l'istruzione `open`;

questa funzione può servire, per esempio, per chiudere il ciclo di cui abbiamo appena parlato.

Un esempio può essere utile. Il seguente programma

```
dim file as string = "/home/vittorio/Documenti/prova.txt"
```

```
open file for output as #1
print #1, "Ciao, Vittorio!"
close #1
```

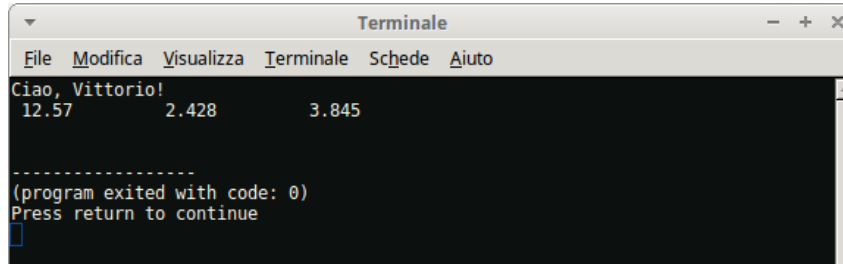
```
open file for append as #1
print #1, 12.57, 2.428, 3.845
close #1
```

```

open file for input as #1
do until eof(1)
    dim contenuto as string
    line input #1, contenuto
    print contenuto
loop
close #1

```

produce il seguente risultato



```

Terminale
File Modifica Visualizza Terminale Schede Aiuto
Ciao, Vittorio!
12.57      2.428      3.845

-----
(program exited with code: 0)
Press return to continue

```

7.4 per la matematica

Quando si richiamano queste funzioni va indicato tra parentesi tonde l'argomento su cui esse devono agire.

L'argomento può essere un numero, una variabile contenente un numero o un'espressione matematica, nel qual caso la funzione agisce sul risultato di questa espressione.

algebriche

`abs()` ritorna il valore assoluto
`exp()` ritorna una potenza del numero e
`log()` ritorna il logaritmo naturale
`sqr()` ritorna la radice quadrata
`fix()` ritorna la parte intera
`frac()` ritorna la parte frazionaria
`int()` ritorna il più alto intero inferiore o uguale
`sgn()` ritorna il segno

Dal momento che FreeBASIC non contiene una definizione preconfezionata della costante e all'occorrenza possiamo memorizzarne un valore ben approssimato con l'istruzione `const e = exp(1)`.

trigonometriche

`sin()` ritorna il seno
`asin()` ritorna l'arcoseno
`cos()` ritorna il coseno
`acos()` ritorna l'arcocoseno
`tan()` ritorna la tangente
`atn()` ritorna l'arcotangente

Gli argomenti per le funzioni trigonometriche vanno espressi in radianti.

Dal momento che FreeBASIC non contiene una definizione preconfezionata della costante π all'occorrenza possiamo memorizzarne un valore ben approssimato con l'istruzione

```
const pi = 2 * acos(0).
```

numeri casuali

randomize() insemiina la generazione di numeri casuali da parte della funzione rnd. Se non si indica l'argomento l'inseminazione avviene sul valore del clock del computer.

rnd ritorna un numero casuale di tipo double compreso tra 0 e 1. Se si usa questa funzione senza prima aver invocato la precedente il numero ritornato sarà sempre quello.

Esempio:

Per scrivere un numero casuale compreso tra 0 e 100 possiamo usare questo codice:

```
randomize
print fix(rnd * 100)
```

8 Interattività con l'utente

Nel piccolo esempio «ciao mondo» che abbiamo visto nel Capitolo 2 e nella sua rivisitazione nel Capitolo 4 il programma conteneva già il dato da elaborare (la scritta "Ciao mondo" nel primo caso direttamente indicata e, nel secondo caso, indicata richiamando la variabile che la conteneva) e, una volta lanciato, forniva il risultato secondo quella scritta. Se ci accontentassimo di questo, per salutare qualche cosa che non sia il mondo ma sia un nostro amico, dovremmo prendere il programma che abbiamo scritto per salutare il mondo, ricopiarlo sostituendo alla parola "mondo" il nome del nostro amico, ricompilarlo e rilanciarlo. Così dovremmo fare per tutti gli amici che volessimo salutare.

Ma con quanto abbiamo visto nel precedente Capitolo 7 possiamo ora fare meglio e modificare il programma in modo che, una volta lanciato, chieda all'utente chi voglia salutare e il programma diventi finalizzato a rivolgere il saluto a chicchessia senza bisogno di essere riscritto tutte le volte.

Il programma diventa il seguente

```
dim nome as string
input "Come ti chiami? ", nome
print "Ciao, "; nome; "!"
sleep
```

Lanciandolo saremo richiesti di indicare il nome di chi salutare e in un attimo vedremo il saluto rivolto a chi abbiamo indicato.

Quest'altro programmino calcola l'area di un triangolo dopo aver chiesto all'utente di indicarne la base e l'altezza

```
dim as double b, h, a
input "indica la base del triangolo: ", b
input "indica l'altezza del triangolo: ", h
a = b * h / 2
? "area: ", a
```

9 Strutture di controllo

Come ogni altro linguaggio di programmazione, FreeBASIC ha dei comandi per condizionare l'esecuzione di certe istruzioni al verificarsi di determinate condizioni oppure per la ripetizione dell'esecuzione di una o più istruzioni.

9.1 Esecuzione condizionale

if

L'istruzione `if`, chiamata istruzione di esecuzione condizionale (in inglese `if` è il nostro `se`), ci dà modo di assoggettare l'esecuzione di un blocco di istruzioni al verificarsi di una determinata condizione: se la condizione è vera viene eseguito il blocco di istruzioni, altrimenti si prosegue l'esecuzione del programma saltando il blocco stesso.

La sintassi è la seguente

```
if <condizione> then
  <istruzioni>
end if
```

dove <condizione> è una qualsiasi espressione che relaziona due valori attraverso operatori di confronto: se la condizione si verifica vengono eseguite la o le istruzioni indicate, altrimenti si passa oltre.

L'istruzione `if` si presta anche all'esecuzione condizionale a due rami. Per ottenere questo dobbiamo abbinarla all'istruzione `else` con questa sintassi

```
if <condizione> then
  <istruzioni>
else
  <istruzioni>
end if
```

In questo caso se la condizione si verifica vengono eseguite le istruzioni contenute nel primo blocco altrimenti vengono eseguite quelle contenute nel secondo blocco dopo `else` (altrimenti). In ogni caso proseguendo poi nell'esecuzione del resto del programma.

Possiamo infine gestire l'esecuzione condizionale a più rami abbinando a `if` l'istruzione `elseif` con questa sintassi

```
if <condizione> then
  <istruzioni>
elseif <condizione> then
  <istruzioni>
elseif <condizione> then
  <istruzioni>
elseif <condizione> then
  <istruzioni>
.....
```

e proseguire quanto vogliamo, chiudendo alla fine con `end if`.

select

Abbiamo appena visto come l'istruzione `if` possa essere applicata a una ramificazione multipla attraverso le istruzioni aggiuntive `elseif`.

L'istruzione `select` può convenientemente essere usata per semplificare la realizzazione della ramificazione multipla nel caso essa dipenda dal confronto tra diverse alternative che una variabile può assumere.

La sintassi è la seguente

```
select case <variabile>
  case <valore>
    <istruzione>
  case <valore>
    <istruzione>
```



```

case <valore>
    <istruzione>
.....
case else
    <istruzione>
end select

```

dove <variabile> può essere di tipo intero o di tipo stringa e <valore> è uno dei valori che la variabile può assumere in corrispondenza al quale eseguire una certa istruzione.

Propongo un esempio nel quale il confronto è basato sul valore di una variabile stringa.

```

dim moneta as string
input "Scegli una moneta: ", moneta
select case moneta
    case "euro"
        print "Europa"
    case "dollaro"
        print "U.S.A."
    case "sterlina"
        print "Regno Unito"
    case "yen"
        print "Giappone"
    case "yuan"
        print "Cina"
    case else
        print "non so"
end select

```

Il programma risponde indicando la zona geografica dove si usa una certa moneta indicata dall'utente e, anche se il programmatore si sforza di prevederle tutte, può sempre succedere che l'utente ne indichi una non prevista: al che il programma risponde «non so».

9.2 Ripetizione

for

Si usa quando si vuole ripetere l'esecuzione di una istruzione o di un blocco di istruzioni per un numero definito di volte.

La sintassi per l'uso di questa istruzione è

```

dim <contatore> as <tipo>
for <contatore> = <partenza> to <arrivo> [step <numero>]
    <istruzioni>
next

```

La variabile che ho battezzato *contatore*, per richiamarne la natura, in genere viene nominata semplicemente *i* e serve per contare le volte che viene eseguito il blocco di istruzioni contenuto tra le indicazioni <partenza> e <arrivo>: dopo le istruzioni, se ne prevede l'incremento di una unità e l'esecuzione termina dopo il raggiungimento del numero indicato come <arrivo>.

L'indicazione *step* è facoltativa e consente di scegliere come si debba scorrere il contatore. Se si omette l'indicazione, la conta avviene per unità (come dire *step 1*).

Questo piccolo programma

```

dim i as short
for i = 1 to 5
    print "Ciao"; " "; i
next i

```

produce il seguente risultato

```
Ciao 1
Ciao 2
Ciao 3
Ciao 4
Ciao 5
```

while

Si usa per ripetere istruzioni fino a quando si verifica una certa condizione.

La sintassi è:

```
while <condizione>
    <istruzioni>
wend
```

Se la condizione è espressa attraverso l'uso di un contatore otteniamo gli stessi risultati che otteniamo con la funzione for vista prima

```
var i = 1
while i <= 5
    print "Ciao"; " "; i
i = i + 1
wend
produce il risultato
```

```
Ciao 1
Ciao 2
Ciao 3
Ciao 4
Ciao 5
```

Ma sono possibili altri meccanismi.

Questo piccolo programma, per esempio, ritorna il carattere che corrisponde al tasto premuto sulla tastiera fino a quando non premiamo il tasto f minuscolo, nel qual caso il programma si arresta.

```
dim s as string
input "premi un tasto ", s
while s <> "f"
    print s
    input "premi un tasto ", s
wend
```

do

Si usa per ripetere istruzioni mentre o fino a quando si verifica una certa condizione.

La sintassi è:

```
do while|until <condizione>
    <istruzioni>
loop
```

Le parole chiave while (mentre) e until (fino a quando) sono di uso alternativo.

Un'applicazione con until l'ho anticipata nel paragrafo 7.3 quando ho mostrato come leggere un file fino a quando se ne raggiunge la fine.

Un'applicazione con `while` può essere questo altro modo di ottenere ciò che abbiamo ottenuto con l'esempio precedente.

```
dim s as string
input "premi un tasto ", s
do while s <> "f"
    print s
    input "premi un tasto ", s
loop
```

10 Grafica

Il FreeBASIC ha una libreria grafica in 2D integrata, in parte compatibile con il QuickBASIC, che permette di gestire semplici primitive grafiche (linee e cerchi), il blitting e caratteristiche aggiuntive non presenti nella libreria grafica originale del QuickBASIC. La libreria non è dipendente dal sistema operativo per cui il codice è portabile da una piattaforma all'altra.

Per default FreeBASIC lavora su terminale non in modalità grafica e per lavorare in modalità grafica occorre creare una finestra grafica che sostituisca il terminale.

10.1 Creare una finestra grafica

La finestra grafica si crea con il comando **screen**.

La sintassi del comando è la seguente:

`screen <modo>, <profondità_colore>`

Ho indicato i due parametri più ricorrenti: altri ce ne sono per usi sofisticati e per quelli rimando al manuale.

`<modo>` rappresenta la risoluzione grafica,

`<profondità_colore>` rappresenta la risoluzione del colore.

risoluzione grafica

Si indica con un numero.

Prescindendo dalla possibilità di indicarla in modo antidiluviano con i numeri 1 e 2 in emulazione CGA, si indica con un numero compreso tra 7 e 21.

I numeri da 7 a 13 corrispondono, insieme ai due precedenti, alle risoluzioni grafiche del vecchio QuickBASIC e contemplano risoluzioni tra 320x200 pixel e 640x480 pixel in emulazione EGA e VGA ormai in disuso.

I numeri da 14 a 21 corrispondono alle risoluzioni originali del FreeBASIC e vanno da 320x240 pixel a 1280x1024 pixel.

Una risoluzione che ritengo ragionevole e più che sufficiente, soprattutto se lavoriamo su un portatile, è la 19, corrispondente a 800x600 pixel. Sapendo che la 20 corrisponde a 1024x768 pixel e la 21 corrisponde a 1280x1024 pixel.

E' possibile creare una finestra di dimensioni personalizzate con l'istruzione **screenres**, la cui sintassi è:

`screenres <larghezza>, <altezza>, <profondità_colore>`

dove, in luogo del `<modo>` previsto dall'istruzione `screen`, indichiamo larghezza e altezza, in pixel, della finestra che ci interessa. Se le dimensioni indicate non sono adatte al computer che stiamo utilizzando avremo messaggi di errore e dovremo cambiare la scelta.

risoluzione del colore

Questo argomento ha effetto solo per le risoluzioni grafiche da 14 a 21 e si indica con uno dei numeri 8, 16 o 32, che sono i bits per pixel destinati a descrivere il colore: con 8 bits arriviamo a differenziare 16 colori, con 32 bits arriviamo a sfumature che neanche Leonardo...

Se non indichiamo l'argomento, per default viene scelto il colore a 8 bits per pixel e potremo scegliere il colore indicandone un riferimento tabellare; se scegliamo 16 o 32 bits per pixel dovremo indicare il colore attraverso la codifica rgb.

Se non indichiamo né argomento né colore la finestra che creiamo con i comandi `screen` o `screenres` è a sfondo nero e accoglie scritte o disegni (primo piano) in bianco.

Per modificare la scelta di default abbiamo a disposizione il comando **color**, la cui sintassi è:

```
color <primo_piano>, <sfondo>
```

dove i due argomenti vengono indicati

- . con un numero da 0 a 15 se non abbiamo scelto la risoluzione del colore o abbiamo scelto la risoluzione colore a 8 bits,

- . con la funzione `rgb(<rosso>, <giallo>, <blu>)`, con `<rosso>`, `<giallo>` e `<blu>` indicati con numeri da 0 a 255 a seconda dell'intensità richiesta, se abbiamo scelto di lavorare con la risoluzione colore a 16 o 32 bits.

Non necessariamente dobbiamo indicare entrambi i parametri. Se vogliamo agire solo sul primo piano indichiamo un solo argomento. Se vogliamo agire solo sullo sfondo indichiamo un solo argomento preceduto da una virgola (,).

L'elenco dei 16 colori con risoluzione a 8 bits è il seguente:

- 0 nero
- 1 blu
- 2 verde
- 3 turchese
- 4 rosso
- 5 rosa
- 6 giallo
- 7 grigio
- 8 grigio scuro
- 9 blu brillante
- 10 verde brillante
- 11 turchese brillante
- 12 rosso brillante
- 13 rosa brillante
- 14 giallo brillante
- 15 bianco

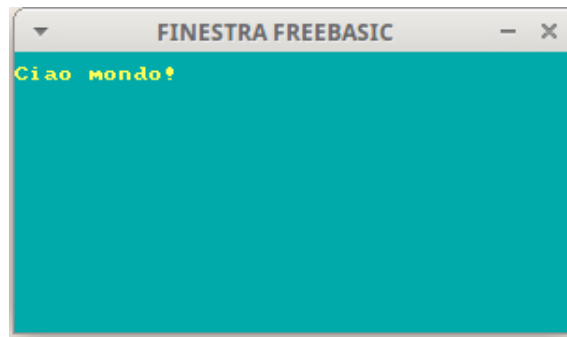
Per l'elenco di una lunga serie di colori corrispondenti a varie combinazioni rgb rimando alla voce Lista dei colori di Wikipedia.

Alla nostra finestra possiamo dare un titolo con l'istruzione `windowtitle(<stringa_titolo>)`.

Il seguente programmino

```
screenres 300, 150
color 14, 3
cls
windowtitle("FINESTRA FREEBASIC")
print
print "Ciao mondo!"
sleep
```

produce la seguente finestra grafica



I codici colore sono indicati con i riferimenti tabellari in quanto, non essendo stata specificata la risoluzione colore con l'istruzione `screenres`, è subentrata la risoluzione di default a 8 bits.

Richiamo l'attenzione sul fatto che, subito dopo aver inserito l'istruzione `color`, è necessaria l'istruzione `cls` (pulisci schermo), per ripulire lo schermo e ripristinarlo con i colori scelti. Inoltre è bene chiudere il programma con l'istruzione `sleep`, in quanto, come avviene in Windows, i programmi grafici partono con un semplice click sul loro nome anche in Linux e questa istruzione, come ho già avuto modo di spiegare, se lanciamo il programma in quel modo, lascia visibile la finestra fino a che non si preme un qualsiasi tasto; in caso contrario la finestra sparirebbe subito.

In questo modo, pur sempre interagendo con il computer attraverso la tastiera, possiamo eseguire tutti i nostri programmi in una finestra grafica anziché sul terminale.

In proposito chiarisco che la libreria grafica integrata in FreeBASIC non è strutturata in modo da produrre interfacce utente grafiche (GUI). A questo scopo possiamo ricorrere al software libero Glade/GTK che ben si integra con FreeBASIC, ovviamente avendo installato sul computer il software Glade e il toolkit GTK.

Altro chiarimento riguarda il fatto che la libreria grafica integrata in FreeBASIC è 2D, cioè bidimensionale. Per la tridimensionalità possiamo ricorrere a OpenGL, che ben si integra con FreeBASIC.

Ma vediamo cosa altro possiamo fare con la libreria integrata.

10.2 Disegnare

Ogni punto (pixel) della finestra grafica è identificato dalle sue coordinate: una in orizzontale, quella che in geometria analitica si usa chiamare *x*, ed una in verticale, quella che in geometria analitica si usa chiamare *y*.

Il punto di coordinate 0 e 0, che chiamiamo origine, è quello in alto a sinistra della finestra.

Attraverso le coordinate possiamo pertanto individuare le posizioni nella finestra e far funzionare alcuni strumenti di disegno rappresentati dalle seguenti istruzioni.

pset

Disegna un punto in una determinata posizione e con un determinato colore. La sintassi è:

`pset (x, y), <colore>`

dove *x* e *y* sono le coordinate orizzontale e verticale e `<colore>` è indicato secondo i modi visti nel precedente paragrafo. Se non si indica il colore il punto viene colorato di bianco.

line

Disegna una linea (o un rettangolo) tra un punto della finestra e un altro. La sintassi è:

`line (x1,y1) - (x2,y2), <colore>`

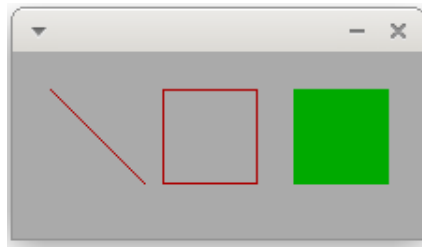
dove (x1, y1) sono le coordinate del punto da dove parte la linea, (x2, y2) sono le coordinate del punto dove arriva la linea e <colore> è indicato secondo i modi visti nel precedente paragrafo. Se non si indica il colore la linea viene colorata di bianco.

Questa istruzione accetta un ulteriore argomento esprimibile con la lettera b, che sta per box o con le lettere bf che stanno per box filled.

Se indichiamo l'argomento b non verrà disegnata una linea ma un rettangolo con vertici opposti nei punti indicati e se indichiamo l'argomento bf il rettangolo verrà riempito del colore indicato.

Il seguente programmino

```
screenres 220, 100
color 4, 7
cls
line (20,20) - (70, 70)
line (80, 20) - (130, 70),, b
line (150, 20)- (200, 70), 2, bf
sleep
produce le seguenti figure
```



Notare la riga dove ho utilizzato l'argomento b, non indicando il colore in modo da usare il rosso già scelto come colore di primo piano: non avendo indicato il colore, ho dovuto inserire una virgola in modo da evidenziare come non utilizzata la posizione riservata al colore in quanto il compilatore, subito dopo l'indicazione dei punti per tracciare la linea, si aspetta l'indicazione o la non indicazione dell'argomento relativo al colore e se vi trova qualche cosa di diverso segnala errore e non procede nella compilazione.

circle

Disegna una circonferenza (o una ellisse o un arco) attorno ad un punto identificato come centro. La sintassi, con gli argomenti base per la circonferenza, è:

```
circle (x, y), <raggio>, <colore>
```

dove x e y sono le coordinate del centro, <raggio> è il raggio e <colore> è indicato secondo i modi già visti.

Immediatamente dopo questi argomenti possiamo indicarne altri due: <partenza> e <arrivo>, attraverso i quali disegnare, in senso anti-orario, un arco di circonferenza a partire dall'angolo indicato, in radianti, come <partenza> fino all'angolo indicato, sempre in radianti, come <arrivo>.

Immediatamente dopo questi ulteriori argomenti è eventualmente inseribile un altro argomento <aspetto> attraverso il quale possiamo disegnare un'ellisse schiacciando la circonferenza ideale in senso orizzontale, indicando un valore dell'argomento inferiore a 1, o verticale, indicando un valore dell'argomento superiore a 1.

Infine, dopo i posti riservati a tutti gli argomenti visti, c'è posto per un ultimo argomento, esprimibile con la lettera f, ad indicare se la figura chiusa (circonferenza o ellisse, non arco) debba essere riempita con il colore precedentemente indicato come colore di primo piano.

Questo programmino

```

screenres 300, 100, 32
color ,rgb(201, 160, 220)
cls
circle (50, 50), 40, rgb(227,11,92)
circle (150, 50), 40, rgb(128, 128, 0),,, 0.5, f
const pi = acos(-1)
circle (250, 50), 40, rgb (0, 0, 255), 0, pi
sleep

```

produce le seguenti figure



Notare come, avendo scelto la risoluzione colore a 32 bits, abbia dovuto indicare i colori con la codifica rgb (color glicine per lo sfondo, color lampone per la circonferenza, color verde oliva per l'ellisse e blu per l'arco di circonferenza).

Sempre ricordando che il compilatore prevede posti riservati ai vari argomenti: da qui, nell'istruzione per disegnare l'ellisse, i due spazi tra virgole ad indicare l'assenza degli argomenti <partenza> e <arrivo> che il compilatore si aspetta, in quelle posizioni, per l'eventuale disegno di un arco.

Il fastidio di dover ricordare tutti questi possibili argomenti con le relative posizioni è compensato dal fatto che, in FreeBASIC, con un paio di istruzioni possiamo disegnare praticamente di tutto.

paint

Consente di colorare l'interno di una figura chiusa conservando un diverso colore per il bordo: cosa che non è possibile fare con l'opzione `f` che abbiamo visto prima. La sintassi è:

```
paint (x,y), <colore_riempimento>, <colore_bordo>
```

dove `x` e `y` sono le coordinate di un punto interno alla figura da colorare, <colore_riempimento> e <colore_bordo> vanno indicati nel solito modo, facendo attenzione che <colore_bordo> coincida con quello indicato per disegnare i contorni della figura.

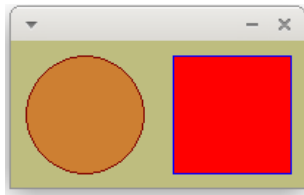
Questo programmino

```

screenres 200, 100, 32
color ,rgb(190,189,127)
cls
circle (50, 50), 40, rgb(128,0,0)
paint (50, 50), rgb(205,127,50),rgb(128,0,0)
line (110, 10)-(190, 90), rgb(0,0,255),b
paint (121, 11), rgb(255,0,0), rgb(0,0,255)
sleep

```

produce le seguenti figure



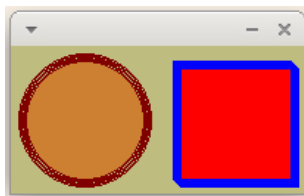
Abbiamo un beige verdastro per lo sfondo, un bordeaux per il bordo del cerchio, un bronzo per l'interno del cerchio, un blu per il bordo del quadrato e un rosso per l'interno del quadrato. Purtroppo i bordi non si vedono molto e, con FreeBASIC non abbiamo un modo diretto per dimensionare le linee.

Ma con un po' d'ingegno si arriva a tutto.

Se vogliamo che i contorni delle figure si vedano bene, per esempio con un bel tratto di 5 pixel, possiamo modificare così il nostro programmino:

```
screenres 200, 100, 32
color ,rgb(190,189,127)
cls
dim i as short
for i = 0 to 5
    circle (50, 50), 40+i, rgb(128,0,0)
next i
paint (50, 50), rgb(205,127,50),rgb(128,0,0)
dim ii as short
for ii = 0 to 5
    line (110+ii, 10+ii)-(190+ii, 90+ii), rgb(0,0,255),b
next ii
paint (121, 17), rgb(255,0,0), rgb(0,0,255)
sleep
```

producendo queste figure



con i bordi evidenziati a dovere, ottenuti disegnando le linee cinque volte, una vicina all'altra, a distanza di un pixel.

draw string

Colloca una scritta in una posizione voluta. La sintassi è:

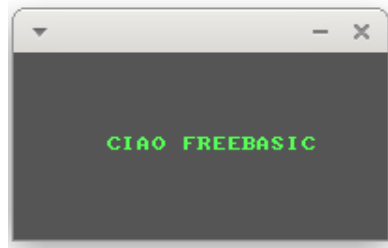
`draw string (x,y), <testo>, <colore>`

dove x e y sono le coordinate del punto che si troverà in alto a sinistra della scritta.

Questo programmino

```
screenres 200, 100
color ,8
cls
draw string (50, 45), "CIAO FREEBASIC", 10
sleep
```

produce questo



10.3 Tracciare

I disegni fatti nel precedente paragrafo sono da riga, squadra e compasso.

FreeBASIC ci offre la possibilità di tracciare altri percorsi, per esempio le curve che rappresentano funzioni matematiche.

Per fare cose di questo tipo abbiamo innanzi tutto bisogno di cambiare il canvas.

La finestra grafica che abbiamo creato nel paragrafo 10.1 è basata su un sistema di coordinate, tutte di segno positivo, con l'origine nel punto in alto a sinistra della finestra.

Ciò non consente di utilizzare anche coordinate di segno negativo come può essere necessario fare in certe situazioni come, per esempio, il tracciamento delle curve di funzioni matematiche.

Esiste una funzione che ci consente di trasformare la nostra finestra grafica in un piano cartesiano, con l'origine degli assi all'interno della finestra in modo che la finestra stessa sia divisa in quattro settori: quello in alto a destra con coordinate entrambe positive, quello in alto a sinistra con coordinata orizzontale negativa e coordinata verticale positiva, quello in basso a sinistra con entrambe le coordinate negative e quello in basso a destra con coordinata orizzontale positiva e coordinata verticale negativa. La sintassi di questa funzione è:

```
window(x1, y1) - (x2, y2)
```

dove la prima coppia di numeri indica il valore delle coordinate del punto in basso a sinistra del piano e la seconda coppia di numeri indica le coordinate del punto in alto a destra.

Se ha da essere un piano cartesiano dobbiamo disegnarne gli assi, con questa sintassi

```
dim x as single
for x = <a> to <b> step 0.001
    pset(x,0), <colore>
    pset(0,x), <colore>
next
```

dove <a> è il maggiore tra i valori assoluti di x1 e y1 e è il maggiore tra x2 e y2.

Il tracciamento del grafico di una funzione avviene con la sintassi:

```
dim x as single
dim y as single
for x = x1 to x2 step 0.001
    y = <f(x)>
    pset (x, y), <colore>
next
```

dove x1 e x2 designano l'intervallo dei valori della x per cui tracciare il grafico, <f(x)> è l'espressione della funzione matematica e <colore> è la solita cosa di cui abbiamo parlato più volte.

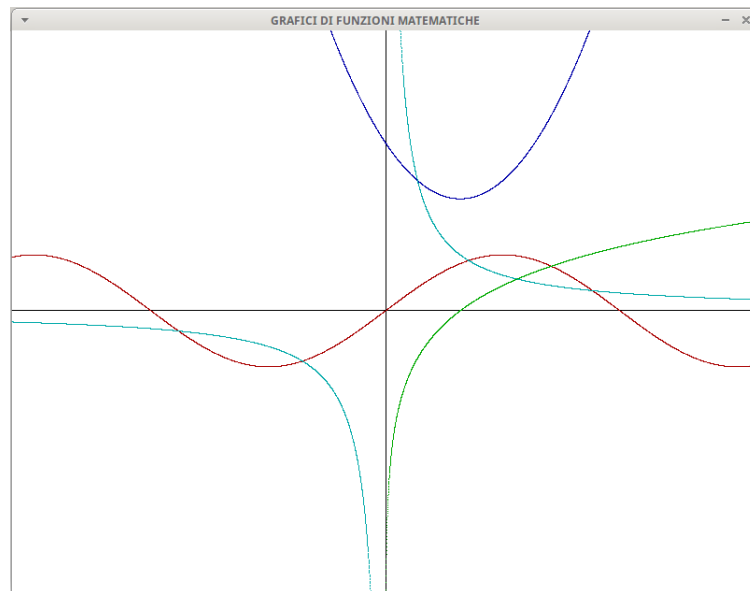
Per riassumere il tutto propongo questo programmino di esempio

```
screen 19
windowtitle ("GRAFICI DI FUNZIONI MATEMATICHE")
color , 15
cls
window(-5,-5)-(5,5)
dim x as single
```

```

dim y as single
for x = -5 to 5 step 0.001
    pset(x,0), 0
    pset(0,x), 0
next
for x = -5 to 5 step 0.001
    y = x^2 - 2*x + 3
    pset (x, y), 1
next
for x = -5 to 5 step 0.001
    y = sin(x)
    pset (x, y), 4
next
for x = -5 to 5 step 0.001
    y = 1/x
    pset (x, y), 3
next
for x = -5 to 5 step 0.001
    y = log(x)
    pset (x, y), 2
next
sleep
    che produce questa finestra

```



Ho scelto un piano cartesiano con origine al centro della finestra e con una scala di grandezze che va da -5 a 5 unità su entrambi gli assi, in modo che l'andamento delle curve nelle vicinanze dell'origine sia ben visibile.

Data la necessità di tracciare in corrispondenza ai valori in numeri reali che assumono le funzioni matematiche, i valori delle x e delle y sono stati dichiarati di tipo single (decimale a precisione singola) e data la dilatazione della scala introdotta ridimensionando una finestra di centinaia di pixel per asse ad una visibilità di poche unità per asse ho scelto uno step per i cicli di tracciamento di 0,001 unità: più questa grandezza è piccola, più nitida e continua è la traccia.

10.4 Rispondere al mouse

FreeBASIC ha due funzioni per rapportarsi al mouse: una per collocare il puntatore del mouse in un determinato punto dello schermo, l'altra per identificare posizione e stato del mouse.

La prima, `setmouse`, colloca il puntatore sullo schermo con la sintassi

```
setmouse (x, y)
```

dove `x` e `y` sono le coordinate del punto in cui collocare il puntatore.

La seconda, `getmouse`, identifica la posizione del mouse con la sintassi

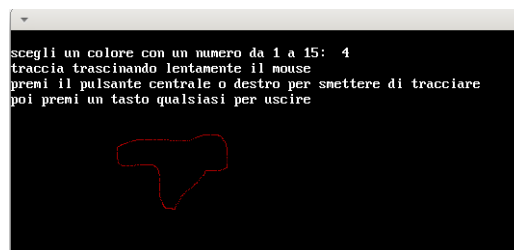
```
getmouse (x, y)
```

dove `x` e `y` sono le variabili in cui la funzione colloca i valori delle coordinate del punto dove si trova il puntatore.

Nella posizione successiva al valore della `y` la funzione identifica lo stato della ruota del mouse e in quella ancora successiva identifica lo stato del pulsante, restituendo, nel mouse a tre pulsanti il valore 0 se è premuto il tasto sinistro, il valore 1 se è premuto il tasto destro e il valore 2 se è premuto il tasto centrale e, nel mouse a due pulsanti, il valore 1 se è premuto il tasto sinistro e il valore 2 se è premuto il tasto destro.

Il seguente codice utilizza queste funzioni per fare un piccolo disegno a mano libera

```
screen 19
dim colore as short
print
input "scegli un colore con un numero da 1 a 15: ", colore
print "traccia trascinando lentamente il mouse"
print "premi il pulsante centrale o destro
      per smettere di tracciare"
print "poi premi un tasto qualsiasi per uscire"
dim leggi_mouse as short
dim x as integer
dim y as integer
dim pulsante as integer
setmouse(200, 150)
do
    leggi_mouse = getmouse(x, y,, pulsante)
    pset(x, y), colore
loop until pulsante = 2
sleep
di cui segue un modestissimo esemplare
```



10.5 Rispondere alla tastiera

Nel paragrafo 7.2 abbiamo già fatto conoscenza con la funzione `inkey`, che acquisisce come stringa il carattere del tasto premuto e può essere utile per fare avvenire qualche cosa alla pressione di un certo tasto che corrisponda ad una certa lettera.

Ma per l'interfacciamento con la tastiera esiste un'altra funzione che può tornare molto utile anche per il riconoscimento di tasti che non rappresentano caratteri. Si tratta della funzione `multikey`, la cui sintassi è:

`multikey(<scancode>)`

dove `<scancode>` è il codice che la tastiera invia al processore per segnalare quale tasto è stato premuto.

L'elenco completo dei codici si trova nel manuale di FreeBASIC sotto la voce `Keyboard Scan Codes`.

In questo elenco i tasti sono individuati da parole che iniziano con le lettere `sc`, seguite dalla lineetta di sottolineatura e dal nome del tasto; di fianco è indicato il codice esadecimale corrispondente.

Con `sc_<lettera>` si identificano i tasti delle lettere.

Tanto per citare i più utili non corrispondenti a lettere dell'alfabeto:

. con `sc_escape` si identifica il tasto `ESC` e il suo codice esadecimale è `&h01`.

. con `sc_left`, `sc_right`, `sc_up`, `sc_down` si identificano i tasti freccia a sinistra, freccia a destra, freccia su e freccia giù, e i rispettivi codici sono `&h4B`, `&h4D`, `&h48` e `&h50`.

. con `sc_space` si identifica la barra spaziatrice e il suo codice è `&h39`.

Per la risposta alla tastiera dobbiamo anche approfondire la conoscenza di una funzione che ho introdotto nel Capitolo 2 e di cui ho parlato anche nel Paragrafo 10.1: la funzione `sleep`.

L'uso di questa funzione senza indicare alcun argomento, come abbiamo fatto negli esempi svolti finora, ferma lo schermo fino a quando viene premuto un qualsiasi tasto e, alla pressione di un qualsiasi tasto, tutto scompare.

La funzione, però, può accettare due utilissimi argomenti.

Il primo argomento è un numero che indica, in millisecondi, la durata dell'attesa: millisecondi significa che 1000 è un secondo.

Il secondo argomento può assumere il valore 0 o il valore 1. Il valore 0, quello di default, ferma l'attesa alla pressione di un tasto qualsiasi; il valore 1 disabilita questa funzione, cioè instaura indifferenza alla pressione dei tasti. L'utilizzo di questo argomento torna necessario quando si voglia utilizzare la tastiera durante un'attesa.

Per esemplificare quanto contenuto in questo paragrafo propongo un semplice programma che ci dà modo di comandare un semaforo, facendogli assumere i colori rosso, giallo o verde alla pressione dei tasti `r`, `g` o `v` e, quando siamo stufi, di uscire dal gioco alla pressione di un certo tasto.



Ho ritenuto utile presentare il programma in due versioni, in una utilizzando la funzione `inkey`, nell'altra utilizzando la funzione `multikey`.

Con la funzione `inkey`:

```
screenres 100,200
sub istruzioni
  print
```

```

    print "r = rosso"
    print "g = giallo"
    print "v = verde"
    print "e = esci"
end sub
istruzioni
do
    if inkey = "r" then circle(50,90), 15, 4,,,f
    if inkey = "g" then circle(50,130), 15, 14,,,f
    if inkey = "v" then circle(50,170), 15, 2,,,f
    sleep 2000, 1
    cls
    istruzioni
loop until inkey = "e"
    Con la funzione multikey:
screenres 100,200
sub istruzioni
    print
    print "r = rosso"
    print "g = giallo"
    print "v = verde"
    print "ESC= esci"
end sub
istruzioni
do
    if multikey(&h13) then circle(50,90), 15, 4,,,f
    if multikey(&h22) then circle(50,130), 15, 14,,,f
    if multikey(&h2F) then circle(50,170), 15, 2,,,f
    sleep 2000, 1
    cls
    istruzioni
loop until multikey(&h01)

```

In entrambi i casi, tenendo premuto uno dei tasti con le lettere r, g o v accendiamo le zone del semaforo dedicate alle luci dei colori rosso, giallo o verde; la luce rimane accesa un paio di secondi e poi scompare; per uscire dal programma dobbiamo premere il tasto e nel primo caso e il tasto ESC nel secondo caso.

10.6 Creare movimento

Il movimento si crea instaurando un ciclo nel quale si disegna in un certo punto dello schermo la figura destinata a muoversi e la si ridisegna dopo breve pausa in una posizione prossima alla precedente cancellando il precedente disegno.

Gli strumenti per fare questo li abbiamo già tutti, serve solo creatività per utilizzarli.

Con il seguente programmino disegniamo un cerchio sullo schermo e lo spostiamo utilizzando le frecce della tastiera

```

screen 19
dim as integer x, y
sub istruzioni
    print "sposta il cerchio con i tasti freccia"
    print "premi ESC per finire"

```

```

end sub
x = 300
y = 200
do
  'verifica se vengono premuti i tasti freccia
  if multikey(&h4B) then x = x - 1
  if multikey(&h4D) then x = x + 1
  if multikey(&h48) then y = y - 1
  if multikey(&h50) then y = y + 1
  'pulisce lo schermo e disegna il cerchio alle nuove coordinate
  cls
  circle(x, y), 30, 4,,,f
  istruzioni
  sleep 10, 1
loop until multikey(&h01)

```

Con quest'altro programmino disegniamo un cerchio che si muove sullo schermo e rimbalza non appena tocca un bordo.

```

screenres 200, 200
dim as single x, y, a, b
x = 100
y = 100
a = 0.5
b = 0.7
do
  print "premi s per finire"
  circle(x,y),10,4,,,f
  if x < 0 then a = - a
  if x > 200 then a = - a
  if y < 0 then b = - b
  if y > 200 then b = - b
  x = x + a
  y = y + b
  sleep 10
  cls
loop until inkey = "s"

```

Il ridisegno del cerchio per creare il suo movimento avviene aumentando, ad ogni ciclo, le coordinate del centro delle grandezze a e b; al raggiungimento di uno dei bordi la grandezza che aumenta la coordinata di quel bordo inverte di segno, in modo che il ridisegno avvenga in direzione contraria.

11 Estensioni

Per tutto ciò che abbiamo fatto finora, e non è poco, ci è bastata la dotazione base del linguaggio FreeBASIC.

Se, presa dimestichezza con il linguaggio, vogliamo fare cose più sofisticate abbiamo a disposizione tantissime librerie esterne.

Ne troviamo un elenco nel manuale FreeBASIC sotto la voce External Libraries Index.

Cliccando sui nomi veniamo sommariamente informati su cosa fa la libreria e ci viene indicato l'indirizzo web ove trovare informazioni complete.

Parte II

Free Pascal

Il linguaggio Pascal fu creato nel 1970 (due anni prima che Dennis Ritchie creasse il linguaggio C), da Niklaus Wirth e Kathleen Jensen a scopo didattico: Wirth era infatti un insegnante di informatica e creò il Pascal per insegnare le basi della programmazione strutturata.

Il nome rende omaggio al filosofo Blaise Pascal, che, tra le tante cose che pensò e scrisse, inventò anche la prima calcolatrice.

La creatura servì per qualche cosa che andava ben oltre la didattica: gran parte dei sistemi operativi per il Macintosh e per Microsoft Windows sono infatti stati scritti in Pascal.

All'epoca esisteva già il linguaggio BASIC, concepito per «beginners», cioè per principianti, ma Wirth non lo riteneva adatto all'insegnamento in quanto era sì facile da imparare ma non aveva strutture dati avanzate e non incoraggiava abbastanza ad analizzare il problema prima di scrivere effettivamente il codice. Non era cioè considerato un linguaggio da programmazione strutturata.

Con l'avvento del PC IBM dotato di hard disk interno, nel 1983, la Società di informatica Borland mette in commercio un compilatore per il linguaggio Pascal, denominato Turbo Pascal, alludendo alla velocità di compilazione.

Nel 1995 Turbo Pascal confluisce nell'ambiente di sviluppo Delphi, sempre in casa Borland.

Nel 1997, Florian Paul Klämpfl, implementando il Pascal della Borland, crea una versione libera del compilatore, denominata FPK Pascal, utilizzando le iniziali del proprio nome.

Vista la natura libera del progetto e il fatto che immediatamente si dedicano al progetto stesso altri collaboratori, il nome del compilatore viene subito convertito in FPC, che sta per Free Pascal Compiler e il linguaggio, pur debitore in toto al linguaggio Turbo Pascal, si chiama Free Pascal.

Nel 1999, Cliff Baeseman, Shane Miller e Michael A. Hess creano il progetto Lazarus, risuscitando (da qui il nome) un fallito progetto di clonazione dell'ambiente di sviluppo Delphi, dotato di un ambiente visuale di progettazione di interfaccia grafica utente.

Grazie a tutto ciò, il mondo del software libero vanta un ambiente di sviluppo di tutto rispetto, multi-piattaforma, in quanto disponibile per tutti i sistemi operativi, che fa dire ai suoi sostenitori «Write once compile everywhere» (Scrivi una volta e compila ovunque) in contrapposizione al motto «Write once run everywhere» (Scrivi una volta e esegui ovunque) dei sostenitori di Java.

La differenza sta nel fatto che i programmi scritti in Java girano ovunque, purché su ciascun computer sia installata la macchina virtuale Java, mentre un programma Pascal/Lazarus compilato su un computer dotato di un certo sistema operativo gira ovunque sia installato quel sistema operativo, anche se sul computer non sono installati né il compilatore Pascal né Lazarus.

In questo manualeto presento quanto serve a un dilettante per capire come funziona il linguaggio Free Pascal e per costruire qualche semplice programma.

A chi si appassioni e voglia approfondire suggerisco l'indirizzo

<http://www.lazaruspascal.it/>,

che è il sito della comunità italiana e vi si trova parecchia documentazione nella nostra lingua.

Il massimo della documentazione, in lingua inglese, si trova su

<https://www.freepascal.org/>.

1 Installazione

Il materiale per creare un ambiente di programmazione Free Pascal lo possiamo trovare in due luoghi.

All'indirizzo <https://www.freepascal.org/>, nella sezione DOWNLOAD, è disponibile il compilatore.

Vi troviamo gli installer, suddivisi per CPU e sistema operativo, dell'ultima versione disponibile nel momento in cui scrivo, la 3.2.2.

Con questa installazione abbiamo disponibile il compilatore e un IDE (ambiente di sviluppo integrato) un tantino antiquato e che ci consente di creare programmi per console, pur abbelliti con una grafica rudimentale. Non abbiamo a disposizione strumenti per creare interfacce utente grafiche (GUI) vere e proprie.

Chi usa Linux, se si accontenta di questi strumenti, può benissimo installare la versione disponibile nell'installatore di software del sistema operativo, denominata fpc.

Se prevediamo di dover creare anche programmi dotati di interfaccia utente grafica ignoriamo quanto detto e andiamo all'indirizzo

<https://www.lazarus-ide.org/index.php?page=downloads>,

dove troviamo quanto serve per disporre dell'ambiente di sviluppo integrato Lazarus per Linux, Windows e Mac. L'ultima versione disponibile è la 4.8 e si appoggia al compilatore 3.2.2.

Anche Lazarus si trova nei repositories delle varie versioni del sistema operativo Linux e lo si potrebbe installare con l'installatore di software. Il ricorso ai file suggeriti dal sito di Lazarus ci offre tuttavia migliori garanzie di fare bene con le dipendenze e di avere un ambiente perfettamente funzionante.

2 Come funziona

I programmi che produciamo con il linguaggio Free Pascal sono eseguibili, cioè funzionano senza alcun interprete anche se sul computer non è installato il compilatore del linguaggio, purché siano stati compilati su un sistema operativo dello stesso tipo di quello del computer su cui lavoriamo: un programma compilato su Windows non gira su Linux o su Mac e viceversa.

La compilazione è la traduzione in linguaggio macchina del programma che abbiamo scritto in Free Pascal, un linguaggio che ha sintassi e regole particolari, che vedremo, ma è di tipo umano, in lingua inglese.

Una volta installato il compilatore, da solo o integrato nell'ambiente di sviluppo Lazarus, il modo più semplice e ruspante di produrre un eseguibile è il seguente:

- . si scrive il programma con un editor di testo,
- . si salva il file con estensione .pas,
- . si apre il terminale (così chiamato nei sistemi Linux e Mac OS X e chiamato prompt dei comandi in Windows),
- . ci si posiziona nella directory dove è stato salvato il file con estensione .pas,
- . si scrive il comando `fpc` seguito dal nome del file con estensione .pas.

Nella directory dove avevamo salvato il file .pas ora troviamo altri due file, uno con lo stesso nome del file .pas ma con estensione .o e un altro con lo stesso nome del file .pas ma senza estensione o, nel sistema operativo Windows, con estensione .exe.

Il file con estensione .o è il file oggetto, file intermedio per effettuare la compilazione eventualmente linkando altri file oggetto, e, al nostro livello dilettantesco, non ci interessa.

Il file senza estensione o con estensione .exe è l'eseguibile ed è il file che, trasferito per semplice copiatura su un altro computer con lo stesso sistema operativo del computer dove è stato compilato, ci consente di far funzionare il nostro programma su quest'altro computer².

Anziché usare un qualsiasi editor di testo per scrivere il programma possiamo usare con profitto l'editor Geany, con una code completion ricca, pur se ispirata a Turbo Pascal.

²La semplice copiatura non basterà per far funzionare il programma in quanto si dovranno superare le eventuali barriere di sicurezza (un file eseguibile può essere un virus). Nel sistema Windows occorrerà seguire le istruzioni per assicurare il sistema che ciò che si sta facendo è regolare. Nei sistemi Linux e Mac OS X, eredi della formidabile sicurezza Unix, occorrerà confermare che si tratta di un file eseguibile nei modi previsti (per esempio, posizionati nella directory dove è stato copiato il file, scrivendo a terminale il comando `chmod 755` seguito dal nome del file).

Per la sua descrizione rimando a quanto ho già detto nel Capitolo 2 della Parte I parlando del FreeBASIC.

Con questi strumenti, conoscendo le basi del linguaggio Free Pascal, possiamo agevolmente creare programmi console, cioè programmi che interagiscono con l'utente attraverso il terminale, quello che in Windows si chiama Prompt dei comandi.

Per creare programmi dotati di interfaccia grafica dobbiamo ricorrere a Lazarus.

Lazarus contiene un ambiente visuale di costruzione dell'interfaccia grafica che ci consente di disegnare l'interfaccia stessa con le sue componenti (pulsanti, finestrelle, ecc.) occupandosi lui di tradurre il disegno in istruzioni per il compilatore. A noi rimane da scrivere, utilizzando le basi del linguaggio, le procedure necessarie per eseguire quanto sottende l'interfaccia stessa, per esempio ciò che deve succedere quando si clicca su un pulsante.

Con Lazarus possiamo ovviamente creare anche semplici programmi da console, ma direi che non vale la pena scomodare Lazarus per così poco e per queste cose semplici facciamo sicuramente meglio con gli strumenti visti prima.

3 Basi del linguaggio

Il compilatore di Free Pascal non distingue tra lettere maiuscole e lettere minuscole, per cui possiamo scrivere le istruzioni in maiuscolo, in minuscolo, con la sola iniziale maiuscola (stile tipico ma non tassativo del linguaggio Pascal classico), a nostro piacimento. La mia personale preferenza è per le lettere minuscole e, nel seguito, mi atterrò a questa preferenza.

Se vogliamo inserire commenti nel codice (frasi e dizioni, che il compilatore non veda, solo per commentare il codice) dobbiamo racchiuderli tra parentesi graffe ({ e }) oppure, se contenuti su una sola linea, antepoendovi il simbolo //.

3.1 Tipi di dato

Free Pascal, buon erede del Pascal standard, è probabilmente il linguaggio di programmazione che contempla la maggiore varietà di tipi di dato e addirittura prevede che il programmatore ne possa costruire di propri.

Per il dilettante alle prime armi mi limito a citare quelli più ricorrenti e con i quali già si può fare molto.

3.1.1 Numeri

Dei tanti tipi numerici previsti, retaggio dei tempi in cui l'hardware disponibile imponeva molta attenzione per evitare spreco di memoria, mi limito a proporre una selezione che ritengo adeguata al principiante e all'hardware moderno.

Numeri interi Per maneggiare numeri interi di dimensione ridotta, di valore inferiore a 65.535, abbiamo a disposizione il tipo **integer**. Una variabile di questo tipo occupa 2 bytes di memoria.

Per maneggiare numeri interi di valore compreso tra -2.147.483.648 e 2.147.483.647 abbiamo a disposizione il tipo **longint**.

Se rinunciamo ad utilizzare il segno possiamo optare per il tipo **longword** ed arrivare a maneggiare numeri di valore compreso tra 0

e 4.294.967.295. Una variabile di questi tipi occupa 4 bytes.

Per i grandi numeri con segno abbiamo a disposizione

. il tipo **int64**

(numeri tra -9.223.372.036.854.775.808 e 9.223.372.036.854.775.807),

. rinunciando al segno, il tipo **qword**

(numeri tra 0 e 18.446.744.073.709.551.615).

Una variabile di questi tipi occupa 8 bytes.

Numeri reali Quando abbiamo a che fare con numeri reali, detti anche a virgola mobile o decimali, ciò che importa maggiormente è la precisione, cioè il numero di cifre significative su cui contare, cifre comprensive di quelle prima e dopo la virgola di separazione (che nel linguaggio Free Pascal è un punto).

Se ci bastano 8 cifre significative abbiamo a disposizione il tipo **single** e la variabile occupa 4 bytes. Se vogliamo 16 cifre significative abbiamo a disposizione il tipo **double** e la variabile occupa 8 bytes.

Possiamo arrivare a 20 cifre significative con il tipo **extended** e la variabile occupa 10 bytes.

Attraverso i tipi qui richiamati, in notazione scientifica, è possibile rappresentare numeri composti da tantissime cifre ma ricordiamo, in ogni caso, che le cifre significative sono quelle prima indicate.

Per esempio, se calcoliamo il fattoriale del numero 32, che è costituito da 36 cifre, utilizzando il tipo intero qword otteniamo un risultato completamente sballato in quanto fuori dal limite rappresentabile con quel tipo (18.446.744.073.709.551.615). Se lo calcoliamo utilizzando il tipo extended otteniamo un risultato che è esatto solo per le prime 20 cifre.

3.1.2 Caratteri

Il tipo **char** può assumere uno dei 256 valori (da 0 a 255) dell'insieme di caratteri ASCII. Tali valori possono essere passati indicando il carattere tra apici semplici ('), indicando il codice decimale del carattere preceduto dal simbolo # oppure indicando il codice esadecimale del carattere preceduto dal simbolo \$.

Il carattere C può essere passato con 'C', #67 oppure \$43.

3.1.3 Stringhe

Il tipo **string** definisce una sequenza di caratteri racchiusa tra apici semplici (').

La sua dimensione massima è di 255 caratteri ma, per risparmiare memoria, si può indicare una dimensione massima inferiore, facendo seguire al nome del tipo, tra parentesi quadre, questa dimensione.

string identifica una stringa di un massimo di 255 caratteri;

string[12] identifica una stringa di un massimo di 12 caratteri;

string[300] genera un errore di compilazione.

Per inserire un apice semplice in una stringa occorre scriverlo due volte (la stringa 'D''Angelo' contiene il nome D'Angelo).

3.1.4 Boolean

Il tipo **boolean** può assumere solo due valori: true e false.

* * *

I tipi visti sopra, ad eccezione dei tipi int64, qword e string, sono i così detti tipi semplici predefiniti del Pascal standard.

I tipi int64 e qword sono stati introdotti da Free Pascal.

Il tipo string è stato introdotto dal Turbo Pascal e da qui è passato a Free Pascal. In realtà è un vettore (array) di elementi di tipo char e, nel Pascal standard veniva gestito così, con molta complicazione. Proprio per evitare queste complicazioni non cito qui i tipi composti, come array, enumerazioni, set, ecc. previsti dal Free Pascal.

3.2 Costanti e Variabili

Le costanti e le variabili sono zone di memoria riservate a contenere i dati oggetto delle elaborazioni previste dal programma.

3.2.1 Costanti

I valori assegnati alle costanti non sono modificabili nel corso del programma.

Una costante si definisce con la seguente sintassi:

```
const <identificatore> = <valore>;
```

dove

<identificatore> è il nome che assegniamo alla costante e serve per richiamarla nel corso del programma,

<valore> è esprimibile con un qualsiasi valore di tipo numerico o di tipo stringa.

Se il valore numerico è superiore al massimo di quello gestibile, solo le sue prime venti cifre vengono registrate esattamente.

Se il valore stringa è superiore a 255 caratteri, solo i primi 255 caratteri vengono registrati.

3.2.2 Variabili

La variabile è destinata a contenere valori che possono essere modificati nel corso del programma.

Una variabile si definisce con la seguente sintassi:

```
var <identificatore>: <tipo>;
```

dove

<identificatore> è il nome che assegniamo alla variabile e serve per richiamarla nel corso del programma,

<tipo> è il tipo di dato che la variabile è destinata a contenere.

Possiamo definire ad un tempo variabili dello stesso tipo con la sintassi:

```
var <identificatore>, <identificatore>, <identificatore>, ...: <tipo>;
```

Infine possiamo inizializzare una variabile con la sintassi:

```
var <identificatore>: <tipo> = <valore>;
```

dove

<valore> deve rispettare il tipo indicato e i suoi limiti.

Bisogna fare molta attenzione alla scelta del tipo con riguardo ai limiti dei dati con esso gestibili.

Infatti, se, indicato un tipo, al momento dell'inizializzazione o nel corso del programma assegniamo alla variabile un valore di un tipo diverso (ad esempio un numero decimale anziché un intero), viene segnalato errore di runtime e il programma si ferma.

Se, invece, indicato un tipo, al momento dell'inizializzazione o nel corso del programma assegniamo alla variabile un valore superiore a quello gestibile dal tipo indicato, avremo risultati completamente sballati senza che lo sappiamo.

Da qui il consiglio di qualcuno: se non abbiamo contezza di quanto grande possa essere un valore da inserire in una variabile, dichiariamo questa variabile di tipo `qword` anche in presenza di un numero intero, in modo da potervi registrare esattamente almeno le sue prime 20 cifre significative.

3.3 Operatori

L'operatore è un simbolo che, inserito tra valori, indicati letteralmente o rappresentati da costanti o da variabili, produce un risultato.

3.3.1 Operatori aritmetici

Si utilizzano tra tipi numerici. I più comuni, elencati in ordine di precedenza, sono:

- * per la moltiplicazione,
- / per la divisione, con l'operando destro diverso da zero,
- div per il quoziente intero tra due numeri interi, con quello di destra diverso da zero,
- mod per il resto della divisione tra due numeri interi, con quello di destra diverso da zero,
- + per l'addizione,
- per la sottrazione.

L'operatore + tra due stringhe le concatena.

3.3.2 Operatori relazionali

Si utilizzano per confrontare tra loro grandezze dello stesso tipo e il risultato del confronto è un valore booleano: true se la relazione è vera, false se la relazione non è vera.

- = uguale a,
- > maggiore di,
- < minore di,
- >= maggiore o uguale a,
- <= minore o uguale a,
- <> diverso da.

3.3.3 Operatori logici

Si utilizzano per operare su variabili di tipo booleano e restituiscono un valore booleano.

- and fornisce il risultato true se entrambi gli operandi sono true,
- or fornisce il risultato false se entrambi gli operandi sono false,
- xor fornisce il risultato true solo se un operando è true e l'altro è false.

3.4 Procedure e funzioni predefinite

La procedura è un comando per svolgere determinate azioni e la funzione è un comando al quale si passano dei valori (detti parametri o argomenti) per ottenere un risultato.

Il linguaggio Free Pascal contiene numerose funzioni predefinite. Le espongo classificandole per tipi di dato.

3.4.1 Funzioni aritmetiche

Consentono la manipolazione di dati numerici.

- abs(n) restituisce il valore assoluto del valore numerico n,
- exp(n) restituisce la potenza ennesima del numero e,
- frac(n) restituisce, sotto forma di numero reale, la parte decimale di n,
- int(n) restituisce, sotto forma di numero reale, la parte intera di n,
- round(n) restituisce, sotto forma di numero intero, la parte intera di n arrotondata,
- trunc(n) restituisce, sotto forma di numero intero, la parte intera di n senza arrotondamenti,
- ln(n) restituisce il logaritmo naturale di n,
- pi restituisce il valore della costante π ,
- sqr(n) restituisce il quadrato di n,
- sqrt(n) restituisce la radice quadrata di n.

Per la potenza ennesima di x possiamo usare la combinata $\exp(\ln(x)*n)$.

Per la radice ennesima di x possiamo usare la combinata $\exp(\ln(x)/n)$.

3.4.2 Funzioni trigonometriche

Esprimendo n in radianti abbiamo le funzioni dirette $\sin(n)$ e $\cos(n)$.

Abbiamo poi $\arctan(n)$ che restituisce, in radianti, il valore dell'angolo avente per tangente n .

Per la tangente possiamo usare la combinata $\sin(n)/\cos(n)$.

Per l'arcoseno possiamo usare la combinata $\arctan(n/\sqrt{1-\text{sqr}(n)})$ e per l'arcocoseno la combinata $\arctan(\sqrt{1-\text{sqr}(n)})/n$.

3.4.3 Funzioni per manipolare stringhe

Forniscono informazioni o esplicitano espressioni relative alle stringhe.

`concat(<stringa>, <stringa>, ...)` concatena stringhe (può convenientemente essere sostituita dall'operatore `+`),

`copy(<stringa>, <posizione>, <quanti>)` copia da `<stringa>`, a partire da `<posizione>`, `<quanti>` caratteri (la posizione va determinata rammentando che i caratteri della stringa sono collocati a partire dalla posizione 0),

`delete(<stringa>, <posizione>, <quanti>)` cancella da `<stringa>`, a partire da `<posizione>`, `<quanti>` caratteri (la posizione va determinata rammentando che i caratteri della stringa sono collocati a partire dalla posizione 0),

`length(<stringa>)` restituisce un intero corrispondente al numero dei caratteri della stringa,

`pos(<cosa>, <stringa>)` restituisce il valore posizionale del primo carattere di una sotto-stringa `<cosa>`.

3.4.4 Generazione di numeri casuali

Per generale un numero casuale compreso in un intervallo occorre inizializzare il generatore con il comando

`randomize`, senza parametri (infatti siamo in presenza di una procedura e non di una funzione),

e poi generare il numero casuale compreso tra 0 e n con la funzione

`random(n+1)`.

3.5 Istruzioni

Le istruzioni costituiscono la parte principale dello sviluppo del programma.

Se si tratta di istruzioni semplici, occupano una sola riga.

Se si tratta di istruzioni composte, da eseguirsi in blocco, esse si elencano tra la parola chiave `begin` e la parola chiave `end`.

L'istruzione deve tassativamente terminare con il punto e virgola `(;)`.

3.5.1 Istruzioni di assegnazione

Si tratta di istruzioni semplici con le quali si assegnano valori alle variabili.

La sintassi è

`<nome_variabile> := <valore>;`

dove

`<nome_variabile>` richiama il nome di una variabile precedentemente definita,

`<valore>` può essere indicato in termini letterali o attraverso una espressione, in ogni caso rispettando rigorosamente il tipo con cui era stata definita la variabile.

Avendo precedentemente definito la variabile `nome` di tipo `string`, l'istruzione `nome := 'Vittorio';` le assegna il valore `'Vittorio'`.

Avendo precedentemente definito la variabile x di tipo `double`, l'istruzione $x := 7.5$; le assegna il valore 7.5.

Con l'istruzione $x := 3 * 4.5$; le assegneremmo il valore 13,5.

Con l'istruzione $x := 2.1 * \exp(\ln(27)/3)$; le assegneremmo il valore 6,3.

3.5.2 Istruzioni condizionali

Si utilizzano per effettuare la scelta tra possibili azioni, a seconda del verificarsi o meno di particolari condizioni.

if

La sintassi è la seguente

```
if (<espressione_logica>) then <istruzione>;
```

dove

<espressione_logica> è la condizione; la sua collocazione tra parentesi tonde non è d'obbligo nel caso di espressione logica semplice (come $a < b$) ma lo diventa nel caso di espressioni logiche complesse (come $(a < b) \text{ and } (c > d)$);

<istruzione> è ciò che deve essere eseguito se la condizione è vera; nel caso di più istruzioni da eseguirsi in blocco vanno elencate tra `begin` e `end`.

Se la condizione non è vera l'istruzione non viene eseguita e si procede con il resto del programma.

Per il caso che la condizione non sia vera possiamo prevedere l'esecuzione di una istruzione alternativa con la sintassi

```
if (<espressione_logica>) then <istruzione> else <istruzione>;
```

Notare che il punto e virgola di fine istruzione va messo solo dopo la seconda <istruzione>. Nel caso di blocchi di istruzioni poste tra `begin` e `end`, il punto e virgola va messo solo dopo l'`end` che termina la seconda <istruzione>.

case

Si utilizza per scegliere un'azione da eseguire in corrispondenza del valore che assume un'espressione.

Rispetto all'istruzione precedente, che prevede solo due possibilità per la scelta (espressione vera o espressione falsa), qui possiamo avere un lungo elenco di valori per orientare le scelte.

La sintassi è

```
case <variabile> of
```

```
  <valore>: <istruzione>;
```

```
  <valore>: <istruzione>;
```

```
  <valore>: <istruzione>;
```

```
  .....
```

```
end;
```

Prima dell'`end` di chiusura dell'istruzione possiamo inserire

```
else <istruzione>;
```

per avere un default nel caso non sia riscontrato alcun <valore> valido.

In questo esempio abbiamo due variabili stringa: `mese`, per il nome italiano di un mese e `traduzione`, per la relativa traduzione in inglese:

```
case mese of
```

```
  'Gennaio': traduzione := 'January';
```

```
  'Febbraio': traduzione := 'February';
```

```
  'Marzo': traduzione := 'March';
```

```

    'Aprile': traduzione := 'April';
    'Maggio': traduzione := 'May';
    'Giugno': traduzione := 'June';
    'Luglio': traduzione := 'July';
    'Agosto': traduzione := 'August';
    'Settembre': traduzione := 'September';
    'Ottobre': traduzione := 'October';
    'Novembre': traduzione := 'November';
    'Dicembre': traduzione := 'December';
else traduzione:= 'non ho capito';
end;

```

Se nella variabile mese inseriamo il valore 'Agosto', nella variabile traduzione viene inserito il valore 'August'. Se inseriamo il valore 'agosto' con la lettera minuscola o 'aosto', nella variabile traduzione viene inserito il valore 'non ho capito', ecc.

3.5.3 Istruzioni di controllo del flusso

Si utilizzano per eseguire più volte, in maniera controllata, un'istruzione o un blocco di istruzioni.

for

Serve quando il ciclo ripetitivo deve essere eseguito un numero predeterminato di volte. La sintassi è:

```

for <contatore>:=<partenza> to <arrivo> do <istruzione>;
dove

```

<contatore> è una variabile destinata a contenere piccoli valori interi, per brevità è in genere battezzata i,

<partenza> è il valore da cui parte la conta,

<arrivo> è il valore in corrispondenza al quale finisce la conta,

<istruzione> è ciò che viene fatto per ogni conta.

Per eseguire 5 volte un'istruzione scriviamo

```

for i := 1 to 5 do <istruzione>;

```

repeat

Serve per eseguire il ciclo ripetitivo fino a quando si verifica un certo evento. La sintassi è:

```

repeat <istruzione> until <condizione>;

```

Anticipando che l'istruzione read(n) serve per leggere un numero digitato sulla tastiera e inserirlo nella variabile n, con quanto segue sommiamo i numeri via via immessi dalla tastiera memorizzando il risultato nella variabile somma e il ciclo termina quando digitiamo 0 come prima cifra.

```

repeat
    read(n);
    somma := somma + n;
until n = 0;

```

4 Programmazione

Un insieme di istruzioni da eseguirsi una di seguito all'altra costituisce un programma e la scrittura di queste istruzioni si chiama programmazione.

Spesso, ed è bene lo sia tutte le volte che è complesso, un programma è suddiviso in tante procedure e funzioni, chiamate anche sottoprogrammi, che vengono richiamate per essere eseguite quando serve.

Abbiamo visto nel precedente Capitolo che alcune procedure e funzioni sono comprese nel pacchetto base di Free Pascal e sono richiamabili senza particolari procedimenti.

Altre le possiamo scrivere noi nello stendere il programma.

Abbiamo anche molte procedure e funzioni raggruppate in librerie aggiuntive al pacchetto base, librerie che in Free Pascal si chiamano `unit`.

Noi stessi possiamo creare librerie per avere a disposizione nostre procedure e funzioni originali anche per altri programmi.

Programmi, procedure e funzioni sono sempre costituiti da una intestazione, da una parte dichiarativa e dal blocco delle istruzioni.

La parte dichiarativa è di fondamentale importanza nel linguaggio Free Pascal e vi si compendia tutta la filosofia della programmazione strutturata: prima di scrivere un programma dobbiamo avere chiaro che cosa vogliamo ottenere in modo da poter dichiarare, ancor prima di scrivere le istruzioni, che cosa ci serve.

Di ciò che abbiamo visto finora la parte dichiarativa deve contenere, nell'ordine:

- . preceduti dalla parola chiave `uses` i nomi delle librerie (`unit`) eventualmente necessarie (non dimenticando di finire la riga con il punto e virgola),
- . le definizioni delle costanti, secondo la sintassi vista nel paragrafo 3.2.1,
- . le definizioni delle variabili, secondo la sintassi vista nel paragrafo 3.2.2.

4.1 Procedure

La procedura è un sottoprogramma che svolge determinati compiti, tra i quali vi possono essere anche calcoli, ma non restituisce direttamente alcun valore.

La sintassi per dichiarare una procedura è la seguente:

```
procedure <nome>;  
  <sezione_dichiarativa>  
begin  
  <istruzioni>  
end;
```

L'indentazione è assolutamente facoltativa e serve per fare in modo che ciò che scriviamo sia umanamente ben leggibile e interpretabile; non influisce assolutamente sulla lettura di ciò che scriviamo da parte del compilatore del programma.

Anticipando che l'istruzione `writeln()` serve per scrivere qualche cosa a terminale, la seguente procedura, richiamata in un programma, scrive tre volte la parola Ciao su righe diverse:

```
procedure scrivi;  
  const messaggio = 'Ciao';  
  var i: integer;  
begin  
  for i := 1 to 3 do writeln(messaggio);  
end;
```

Nella parte dichiarativa abbiamo inizializzato come costante il messaggio da scrivere e abbiamo dichiarato la variabile `i` che ci servirà da contatore per il ciclo `for` e nelle istruzioni abbiamo inserito il ciclo per scrivere tre volte il messaggio.

Nel corso del programma basterà chiamarla con l'istruzione `scrivi`; per ottenere la scrittura dei messaggi.

La costante e la variabile dichiarate nella sezione dichiarativa della procedura vengono generate in memoria al momento della chiamata nel programma e, immediatamente dopo l'esecuzione della procedura, lo spazio allocato in memoria viene liberato.

4.2 Funzioni

La funzione è un sottoprogramma che ritorna un valore del tipo previsto nell'intestazione e denominato come la funzione.

La sintassi per dichiarare una funzione è la seguente:

```
function <nome> (<parametro>:<tipo>;...) :<tipo>;  
  <sezione_dichiarativa>  
  begin  
    <istruzioni>  
  end;
```

I parametri sono i valori necessari perché la funzione svolga il proprio compito e vanno indicati, nell'ordine previsto e tra parentesi tonde, quando si chiama la funzione.

Tra le istruzioni non potrà mancare quella di assegnazione del valore restituito dalla funzione alla variabile avente lo stesso nome della funzione.

La seguente funzione calcola l'area del triangolo avente una certa base e una certa altezza:

```
function area_triangolo(base:double;altezza:double):double;  
  begin  
    area_triangolo := base*altezza/2;  
  end;
```

Nel corso del programma, richiamandola, per esempio, con l'istruzione `area_triangolo(3,2)`, restituisce il valore 3, che è l'area di un triangolo di base 3 e altezza 2.

Richiamata con l'istruzione `area_triangolo(7.5, 13.2)` restituisce il valore 49,5.

4.3 Unit

Le procedure e le funzioni che costruiamo con la sintassi vista nei precedenti paragrafi possono essere inserite come tali nel listato del programma.

Esiste tuttavia la possibilità di raccogliere procedure e funzioni in librerie esterne al programma, in modo che siano utilizzabili in qualsiasi programma, semplicemente dichiarando la volontà di usarle. In Free Pascal queste librerie si chiamano unit.

Possiamo costruire noi stessi delle unit con la sintassi:

```
unit <nome>;  
  interface  
    <prototipi>  
  implementation  
    <procedure e funzioni>  
end.  
dove
```

<prototipi> è l'elenco delle intestazioni delle procedure e delle funzioni che inseriamo nella unit e costituiscono l'<interface>,

<implementation> contiene le procedure e le funzioni costruite con le sintassi viste nei precedenti paragrafi.

Una volta scritto il file di programmazione della unit con un editor di testo, lo salviamo con lo stesso nome dato alla unit e con l'estensione `.pas` e lo compiliamo nello stesso modo con cui si compilano i programmi, come abbiamo visto nel Capitolo 2 di questa Parte II.

Otteniamo così due nuovi file aventi lo stesso nome del file `.pas`, l'uno con estensione `.o` e l'altro con l'estensione `.ppu`.

Questi due file vanno tenuti a disposizione nella directory dove scriviamo il programma che richiama la unit oppure, meglio, vanno archiviati nella posizione dove verranno altrimenti cercati dal compilatore del programma che richiama la unit:

`/usr/lib/fpc/3.2.0/units/` o similare in Linux e Mac,

`C:\FPC\3.2.0\units` o similare in Windows con installato solo il compilatore,

`C:\lazarus\units` in Windows con installato Lazarus.

Per esempio, Free Pascal non ha una funzione predefinita per calcolare il fattoriale di un numero. Se prevediamo di avere spesso bisogno di utilizzare questa funzione nei nostri programmi possiamo creare una libreria che la contenga e, in previsione di arricchirla con altre funzioni matematiche, chiamiamola `matematica`.

```
unit matematica;
interface
    function fattoriale(n: integer): extended;
implementation
    function fattoriale(n: integer): extended;
    begin
        if n = 1 then fattoriale := 1
        else fattoriale := n * fattoriale(n-1);
    end;
end.
```

Scrivendo i programmi in cui vogliamo utilizzare questa funzione, dichiariamo l'uso della nostra libreria nella parte dichiarativa con

```
uses matematica;
```

e potremo richiamare la funzione `fattoriale()` ivi contenuta quante volte vogliamo, passandole un numero intero come parametro tra le parentesi tonde.

Avendo scelto come tipo del risultato `extended`, esso sarà espresso in notazione scientifica con le sole prime 20 cifre esatte.

4.4 Programma

Un programma è il risultato più compiuto dell'attività di programmazione quando questa, attraverso la compilazione, sfocia nella produzione di un file eseguibile.

Nei precedenti tre paragrafi abbiamo visto attività di sottoprogrammazione e di supporto alla programmazione, attività che, anche se, come per la unit, le sottoponiamo a una compilazione, non sfociano nella produzione di un file eseguibile.

La sintassi per produrre un programma è la seguente:

```
program <nome>;
    <sezione_dichiarativa>
begin
    <istruzioni>
end.
```

Notare come la sintassi sia la stessa prevista per la procedura, con la sola differenza che dopo la parola finale `end` abbiamo un punto fermo (.) anziché un punto e virgola (;).

5 Interattività con l'utente

Non necessariamente, ma quasi sempre, il funzionamento di un programma prevede interattività con l'utente: normalmente, infatti, un programma per computer prevede l'inserimento e la stampa di messaggi e dati.

L'interfacciamento a questo scopo può avvenire utilizzando il terminale che si apre sullo schermo del computer e la tastiera del computer stesso (in tal caso si parla di programma per console) oppure attraverso una apposita finestra grafica creata sullo schermo del computer ed attrezzata in modo che vi si possa agire anche utilizzando il mouse (in tal caso si parla di programma con GUI, Graphical User Interface).

5.1 Programmi console

Possiamo scriverli con l'editor di Lazarus scegliendo da menu FILE ▷ NUOVO ▷ PROGETTO ▷ PROGRAMMA SEMPLICE o FILE ▷ NUOVO ▷ PROGETTO ▷ PROGRAMMA.

Personalmente, come ho detto nel Capitolo 2 di questa Parte II, ritengo meglio utilizzare l'editor Geany.

5.1.1 Output

Si realizza con le istruzioni `write()` e `writeln()`.

La differenza tra le due sta nel fatto che la prima scrive e non va accapo mentre la seconda scrive e fa un salto a nuova linea.

Tra le parentesi si indica cosa scrivere. Se si omettono le parentesi o non vi si inserisce nulla, nel caso di `writeln` inserito tra altre istruzioni `writeln`, si realizza una riga vuota.

Il cosa scrivere può essere una stringa scritta tra apici semplici, un valore numerico scritto in termini letterali, il valore di una variabile richiamandone il nome, il risultato di una espressione matematica. E' possibile indicare più cose da scrivere sulla stessa riga separandole con una virgola.

Data l'esistenza di una variabile `x` contenente il valore 5 e di una variabile `s` contenente il valore Vittorio,

```
writeln('Ciao'); scrive Ciao
writeln(s); scrive Vittorio
writeln('Ciao, ', s); scrive Ciao, Vittorio
writeln(12); scrive 12
writeln(7/2); scrive 3.5
writeln(x); scrive 5
writeln('x vale: ', x); scrive x vale 5
writeln('3 + 7 fa ', 3+7); scrive 3 + 7 fa 10
```

Esiste la possibilità di formattare output numerici indicando gli spazi dedicati per il relativo posizionamento e le cifre decimali da evidenziare, con la sintassi

```
writeln(<valore>: <spazi>: <decimali>);
```

Le istruzioni consecutive

```
writeln(pi:10:5);
writeln(12.7657485768:10:5);
scrivono i numeri così posizionati e dimensionati
   3.14159
  12.76575
```

5.1.2 Input

Per incamerare dati digitati sulla tastiera abbiamo le istruzioni `read()` e `readln()`.

La differenza tra le due sta nel fatto che la prima legge e non va accapo mentre la seconda legge ed effettua un salto a linea nuova.

Tra le parentesi si indica il nome della variabile in cui va inserito il dato.

L'istruzione `readln` seguita direttamente dal punto e virgola si può utilizzare per provocare un'attesa indefinita durante l'esecuzione del programma, in attesa della pressione di un tasto.

Tra le parentesi si possono indicare più nomi di variabili separati da virgola. In tal caso sulla tastiera occorre digitare i relativi valori separati da almeno uno spazio.

All'istruzione `read()` o `readln()` è opportuno far precedere un'istruzione `write()` o `writeln()` contenente la descrizione dell'input richiesto.

5.1.3 Semplici programmi console di esempio

Area e circonferenza di un cerchio

Non ci serve richiamare alcuna libreria. Le variabili che ci interessano sono il raggio del cerchio per l'input, l'area e la circonferenza per l'output e, avendo a che fare con grandezze ragionevolmente limitate esprimibili con numeri reali, ci basta siano di tipo `double`.

Il listato del programma potrebbe essere questo:

```
program cerchio;
var
    raggio, area, circonferenza: double;
begin
    write('Inserisci il raggio del cerchio: ');
    readln(raggio);
    area := sqr(raggio) * pi;
    circonferenza := raggio * 2 * pi;
    writeln('area: ', area:0:3);
    writeln('circonferenza: ', circonferenza:0:3);
    write('Premi INVIO per terminare');
    readln;
end.
```

I risultati vengono espressi con tre cifre decimali nella posizione del cursore (formattazione `:0:3`). Le ultime due righe del programma sono necessarie, se usiamo il sistema operativo Windows, per mantenere aperta la finestra del prompt dei comandi fino alla pressione del tasto INVIO. In mancanza di ciò, lanciato il programma richiamando l'eseguibile o cliccandoci sopra, il prompt dei comandi, eseguito il programma, si chiuderebbe senza lasciarci il tempo di vedere i risultati delle elaborazioni.

Permutazioni

Nel calcolo combinatorio le permutazioni corrispondono al fattoriale del numero degli elementi da permutare.

Se abbiamo costruito la unit matematica come previsto nel Paragrafo 4.3 e l'abbiamo archiviata a dovere, il listato del programma che calcola le permutazioni possibili con `n` elementi potrebbe essere il seguente:

```
program permutazioni;
uses matematica;
var n: integer;
begin
    write('Numero degli elementi: ');
    readln(n);
    writeln(fattoriale(n):0:0);
    write('Premi INVIO per terminare');
    readln;
```

end.

La formattazione del risultato (:0:0) è per evitare l'esposizione del risultato stesso in notazione scientifica come avverrebbe, dato il tipo attribuitogli nella unit *matematica*.

Se non abbiamo a disposizione la unit, possiamo inserire la funzione che calcola il fattoriale nel programma:

```
program permutazioni;
var n: integer;
function fattoriale(n: integer): extended;
begin
    if n = 1 then fattoriale := 1
    else fattoriale := n * fattoriale(n-1);
end;
begin
    write('Numero degli elementi: ');
    readln(n);
    writeln(fattoriale(n):0:0);
    writeln
    write('Premi INVIO per terminare');
    readln;
end.
```

Notare come il sottoprogramma (la funzione) che calcola il fattoriale preceda il blocco del vero e proprio programma, che deve sempre essere l'ultimo scritto.

Saluto personalizzato

Qui il programma chiede il nome all'utente per poterlo salutare come si deve.

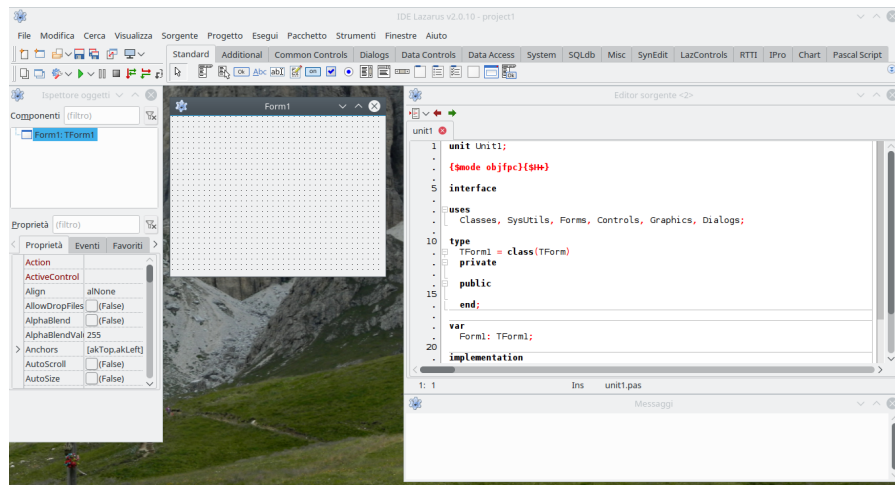
```
program saluto;
var nome: string;
begin
    write('Come ti chiami? ');
    readln(nome);
    writeln('Ciao, caro ', nome, '!');
    write('Premi INVIO per finire');
    readln;
end.
```

5.2 Programmi con GUI

Per realizzare programmi con interfaccia utente grafica usiamo Lazarus.

Al suo lancio, Lazarus si appresta a questo compito, corrispondente alla scelta di menu FILE ► NUOVO ► PROGETTO ► APPLICAZIONE.

L'area di lavoro si presenta così

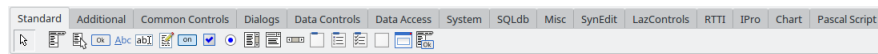


In alto abbiamo innanzi tutto la barra dei menu per la scelta delle varie cose che vogliamo fare. Appena sotto, sulla sinistra, abbiamo due piccole barre di strumenti sovrapposte



che facilitano l'accesso ai compiti più ricorrenti attraverso click su icone. Passando il mouse sulle icone stesse siamo informati sullo strumento che rappresentano.

Sulla destra di queste due barre abbiamo la zona dedicata alla scelta dei componenti l'interfaccia che vogliamo costruire



I componenti, altrimenti detti widgets, sono raggruppati in quindici schede.

Per default è aperta la prima, denominata STANDARD, che contiene i componenti più comuni, quelli che sicuramente bastano e avanzano per un principiante.

Sotto la linea per la scelta della scheda si allineano le icone dei componenti appartenenti alla scheda aperta. Le icone sono autoesplicative, comunque, passando il mouse su di esse, ne vediamo la definizione.

Se esploriamo le varie schede ci rendiamo conto di quanto sia vasta la strumentazione di Lazarus.

Sotto questa zona sono collocate le finestre di lavoro.

Sulla sinistra abbiamo l'ISPETTORE OGGETTI, suddiviso in due finestre: quella in alto presenta l'elenco dei componenti impegnati nella nostra interfaccia in modo che li possiamo tenere sotto controllo e selezionarli cliccando sopra il loro nome con il mouse. Quella in basso (scheda PROPRIETÀ) elenca le proprietà del componente selezionato in una struttura atta ad intervenire per modificarle a nostro piacimento. Vi possiamo aprire anche la scheda EVENTI.

Sulla destra abbiamo l'EDITOR SORGENTE, nel quale lo stesso Lazarus inserisce il codice corrispondente alla realizzazione dell'interfaccia con le componenti scelte e il programmatore è chiamato ad inserire il codice delle procedure e delle funzioni per svolgere i compiti che gli eventi captati dall'interfaccia richiedono.

Al centro abbiamo il componente base dell'interfaccia grafica, il Form, cioè la finestra in cui inserire gli altri vari componenti con cui costruire l'interfaccia stessa. Possiamo dimensionarla a piacimento agendo per trascinamento con il mouse sui bordi e sugli angoli.

Con questa strumentazione, se abbiamo padronanza delle basi del linguaggio Free Pascal, ci basta acquisire dimestichezza con i meccanismi di Lazarus per realizzare interfacce grafiche anche

impegnative. Questi meccanismi è difficile spiegarli in un manuale ma, provando e riprovando con pazienza e grazie all'intuitività su cui si basa Lazarus, non occorre molto per arrivare a destreggiarsi.

Alla base di tutto sta il concetto che i componenti dell'interfaccia che andiamo a costruire sono informaticamente degli oggetti:

- . dotati di proprietà, alle quali possiamo accedere con istruzioni composte dal nome dell'oggetto seguito, separato da un punto (.), dal nome della proprietà,
- . collegabili ad un evento da cui si possano far dipendere conseguenze: che ci si passi sopra con il mouse, che ci si clicchi sopra, ecc.

5.2.1 Output

Il più comodo componente per l'output in una GUI è la Label (tipo `TLabel`), che automaticamente, in ordine di inserimento nel Form, prende i nomi `Label1`, `Label2`, ecc.

Essa si presta per scrivere nel form, in posizioni volute, titoli, istruzioni, ecc. oppure per scrivere il risultato di elaborazioni.

La proprietà che consente di inserire del testo nella Label è `Caption`.

Con la sintassi

```
labelx.Caption := <valore>;
```

inseriamo nella Label numero x un valore, come se la Labelx fosse una variabile del linguaggio Pascal di base.

Questo valore può essere solo una stringa o una concatenazione di stringhe (`<stringa> + <stringa> + ...`).

Valori numerici si inseriscono convertiti con `inttostr()` o `floattostr()` a seconda del tipo.

Esiste una funzione per formattare un numero decimale convertito in stringa:

```
floattostrf(<valore>, <formato>, <precisione>, <decimali>)
```

dove

`<valore>` è il numero o l'espressione da convertire,

`<formato>` è il tipo di conversione: i più interessanti sono

`ffgeneral` per ottenere un numero non in notazione scientifica,

`ffnumber` per ottenere lo stesso con la separazione delle migliaia,

`<precisione>` è il numero di cifre significative,

`<decimali>` è il numero di cifre decimali (con arrotondamento dell'ultima).

```
floattostrf(<valore>, ffnumber, 20, 3)
```

scrive `<valore>` in formato con separatori delle migliaia (,) e 3 decimali.

5.2.2 Input

Il classico componente per acquisire dati in una GUI è l'Edit (tipo `TEdit`), che automaticamente, in ordine di inserimento nel Form, prende i nomi di `Edit1`, `Edit2`, ecc.

Si tratta di una finestra nella quale si scrive un dato digitandolo sulla tastiera. La proprietà che registra ciò che si scrive nella finestra è `Text`.

Con la sintassi

```
<variabile> := editx.Text;
```

acquisiamo in una variabile il valore inserito nella finestra numero x, che è una stringa.

Se la variabile è di tipo numerico occorre usare le funzioni di conversione `strtoint()` o `strtofloat()` a seconda del tipo.

```
<variabile> := strtoint(editx.Text);
```

 inserisce quanto letto come numero intero,

```
<variabile> := strtofloat(editx.Text);
```

 inserisce quanto letto come numero decimale.

5.2.3 Semplici programmi con GUI di esempio

Cominciamo a vedere come si imposta un progetto con Lazarus.

Creiamo innanzitutto una cartella, dove ci è più comodo, destinata a contenere i file del progetto e la chiamiamo `mio_progetto` (nel caso specifico al posto di `mio_progetto` mettiamo un nome evocativo del contenuto del progetto).

Lanciamo Lazarus.

Lazarus ha l'abitudine di aprirsi sull'ultimo progetto sviluppato. Per affrontare un nuovo progetto ricorriamo al menu **FILE** ▸ **NUOVO** ▸ **PROGETTO** ▸ **APPLICAZIONE**. Ci troviamo così di fronte l'area di lavoro illustrata a pagina 95.

Da menu scegliamo **FILE** ▸ **SALVA COME...** e salviamo il primo file che ci viene proposto, quello con estensione `.lpi`, dandogli il nome che abbiamo assegnato al progetto (`mio_progetto` o quello più evocativo che abbiamo scelto) e mantenendo l'estensione `.lpi`. Salviamo poi il secondo file che ci viene proposto, denominato `unit1.pas` così com'è.

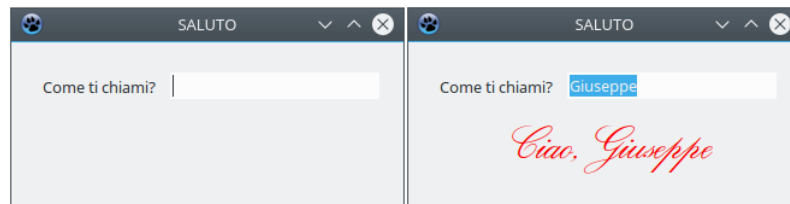
Ora siamo pronti a sviluppare il nostro progetto raccogliendo il nostro lavoro in maniera ordinata nella cartella dedicata.

Nel corso del lavoro è bene salvare ricorrentemente ciò che abbiamo fatto ricorrendo al menu **FILE** ▸ **SALVA TUTTO**.

Saluto personalizzato

E' la versione GUI dell'ultimo esempio svolto nel Paragrafo dedicato ai programmi console: il computer chiede all'utente il suo nome e lo saluta.

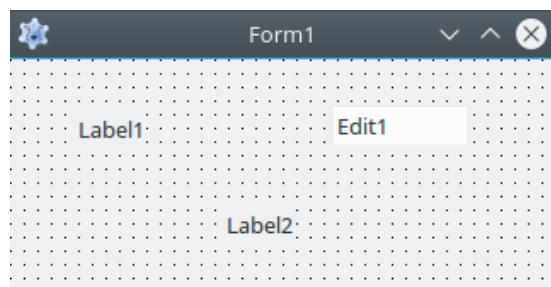
Il programma si apre come da schermata di sinistra. Premendo **INVIO** dopo aver inserito il nome nella finestra, digitandolo sulla tastiera, la schermata diventa quella di destra. Il programma si chiude cliccando sulla **X** in alto a destra.



Impostato il progetto come visto sopra, prima cosa è disegnare l'interfaccia.

Nell'esemplare di questa che troviamo al centro dell'area di lavoro inseriamo le componenti che ci servono: una finestra Edit (icona ) e due Label (icona ). Selezionata l'icona nella barra dei componenti clicchiamo sul form nella posizione dove inserire.

Il form diventa



Diamo un titolo al form:

. selezioniamo il form cliccando sul suo nome nell'elenco dei componenti dell'ISPETTORE OGGETTI oppure cliccando in una zona libera del form stesso,

. scorriamo nella zona delle proprietà dell'ISPETTORE OGGETTI fino a trovare la proprietà CAPTION e nella finestrella a destra del nome scriviamo SALUTO.

Ora dedichiamoci ai particolari dell'interfaccia.

Utilizziamo la Label1 per il messaggio di richiesta del nome dell'utente che vuole farsi salutare:

. selezioniamo la Label1 cliccando sul suo nome nell'elenco dei componenti di ISPETTORE OGGETTI oppure cliccando su di essa nel form,

. nella zona delle proprietà dell'ISPETTORE OGGETTI selezioniamo la proprietà CAPTION e nella finestrella a destra del nome scriviamo Come ti chiami?

Utilizziamo la finestrella Edit1 per far immettere il nome all'utente:

. selezioniamo la Edit1 cliccando sul suo nome nell'elenco dei componenti di ISPETTORE OGGETTI oppure cliccando su di essa nel form,

. scorriamo nella zona delle proprietà dell'ISPETTORE OGGETTI fino a trovare la proprietà TEXT e cancelliamo ciò che c'è scritto nella finestrella a destra del nome,

. trascinando il mouse con premuto il tasto sinistro sui bordi sinistro e destro del componente diamogli una dimensione acconcia (per esempio che vada dal termine della Label1 fino al bordo del form).

Sempre lavorando di trascinamento con il mouse posizioniamo Label1 e Edit1 in modo allineato.

Ci rimane la Label2, che riserviamo all'inserimento del saluto:

. selezioniamo la Label2 cliccando sul suo nome nell'elenco dei componenti dell'ISPETTORE OGGETTI oppure cliccando su di essa nel form,

. selezioniamo la proprietà AUTOSIZE nella zona delle proprietà di ISPETTORE OGGETTI e disattiviamo l'opzione TRUE nella finestrella a destra del nome in modo che diventi FALSE,

. lavorando di trascinamento del mouse sui bordi del componente allarghiamo la zona della Label in modo che occupi tutto lo spazio restante del form sotto i componenti sistemati prima,

. nella zona delle proprietà dell'ISPETTORE OGGETTI selezioniamo la proprietà CAPTION e cancelliamo il contenuto della finestrella a destra del nome,

. nella zona delle proprietà dell'ISPETTORE OGGETTI selezioniamo la proprietà ALIGNMENT e cliccando nella finestrella a destra del nome, scegliamo TACENTER,

. infine scegliamo un font per la scritta del saluto: sempre nella zona delle proprietà dell'ISPETTORE OGGETTI scorriamo fino a selezionare la proprietà FONT e apriamo il relativo menu cliccando sul pulsante con tre puntini a destra del nome; nella finestra di dialogo scegliamo font e dimensioni (nel mio caso ho scelto RegencyScriptFLF, che ho la fortuna di avere sul computer dove ho lavorato, dimensione 32). Se apriamo le sotto proprietà della proprietà FONT cliccando sulla freccina a sinistra del nome, possiamo selezionare COLOR e, cliccando sul pulsante con tre puntini a destra del nome, aprire la finestra di dialogo per scegliere il colore con cui scrivere il saluto (nel mio caso ho scelto un rosso vivo).

Abbiamo così realizzato visivamente la parte grafica dell'interfaccia utente e Lazarus si è occupato di tradurla in istruzioni leggibili dal compilatore.

A questo punto dobbiamo inserire il codice per ciò che si deve realizzare con la parte grafica.

Nel nostro caso si tratta di acquisire ciò che l'utente scrive nel componente Edit1 e utilizzarlo per scrivere la stringa del saluto da inserire nel componente Label2 e prevedere un evento che dia avvio a questa azione: nel nostro caso abbiamo scelto che l'evento sia la pressione di INVIO dopo che è stato scritto il nome della persona da salutare nel componente Edit1 (Lazarus definisce questo evento EDITINGDONE).

Selezioniamo allora, nella maniera che abbiamo ormai imparato, il componente in cui si deve verificare l'evento, che è Edit1, e nella finestrella in basso dell'ISPETTORE OGGETTI apriamo la scheda EVENTI: scorriamo fino a trovare ONEDITINGDONE, selezioniamolo cliccandoci sopra e clicchiamo sul pulsante con tre puntini che compare a destra del nome.

Veniamo così posizionati con il cursore nella finestra dell'editor nel punto in cui inserire le istruzioni nel codice della procedura relativa all'evento selezionato, già predisposta da Lazarus: tra begin e end, dove lampeggia il cursore, scriviamo

```
label2.Caption:='Ciao, ' + edit1.Text;
```

La code completion di Lazarus ci aiuta a farlo.

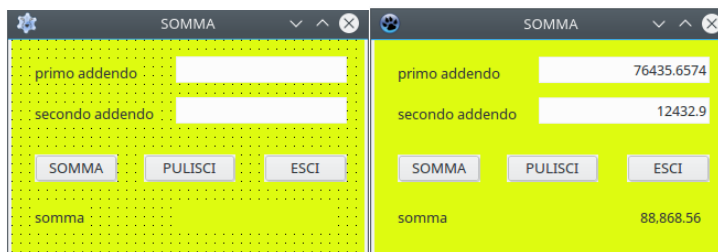
Da menu FILE ► SALVA TUTTO salviamo il nostro lavoro.

Da menu ESEGUI ► COMPILA compiliamo e produciamo l'eseguibile, che troveremo nella cartella dove abbiamo salvato il progetto: è il file che porta il nome del progetto senza alcuna estensione (o estensione .exe se siamo in Windows).

Somma

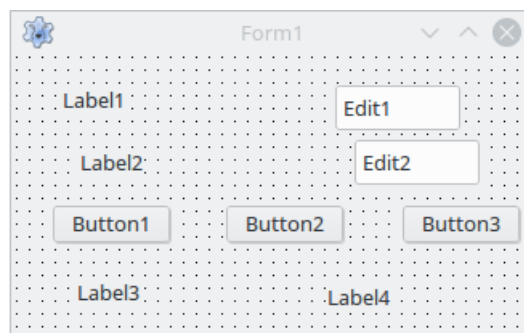
Un programma facile facile che esegue semplicemente l'addizione di due addendi, nel quale vediamo applicate un po' di cose utili.

La schermata del programma è questa



A sinistra abbiamo la schermata di avvio e a destra la schermata dopo che è stata eseguita l'addizione tra i due addendi inseriti nelle apposite finestrelle: il risultato è espresso con un formato che evidenzia le migliaia e approssima a due cifre decimali.

I componenti sono i seguenti



Rispetto all'esempio precedente abbiamo la novità dei pulsanti (Button nella terminologia di Lazarus), che corrispondono al componente con l'icona , e del colore di sfondo del form.

Il colore di sfondo del form si ottiene selezionando il form e, scorrendo le proprietà, scegliendo Color: agendo sul menu che si apre cliccando sul pulsante con i tre puntini a destra del nome della proprietà selezionata scegliamo il colore che più ci piace.

Con le tecniche viste nel precedente esempio dedichiamo Label1 alla descrizione 'primo addendo' e Edit1 al suo inserimento, dedichiamo Label2 alla descrizione 'secondo addendo' e Edit2 al suo inserimento, dedichiamo Label3 alla descrizione della riga del risultato 'somma' e Label4 a mostrare il risultato della somma.

I pulsanti li dedichiamo a tre compiti diversi: il primo ad eseguire la somma, il secondo a ripulire l'interfaccia dai dati di una somma precedente, il terzo ad uscire dal programma. Per

scrivere sul pulsante un titolo evocativo del compito ('somma', 'pulisci' ed 'esci') selezioniamo il pulsante e nelle proprietà, scriviamo il titolo nella finestrella della proprietà CAPTION.

Ora dobbiamo far corrispondere ai tre pulsanti le procedure per i compiti assegnati.

Selezionato Button1, apriamo la scheda Eventi dell'Ispettore Oggetti e selezioniamo l'evento OnClick. Nell'editor facciamo in modo che la procedura abbozzata da Lazarus, inserendo i dati in rosso, diventi la seguente:

```
procedure TForm1.Button1Click(Sender: TObject);
    var a,b: double;
begin
    a:=strtofloat(edit1.Text);
    b:=strtofloat(edit2.Text);
    label4.Caption:=floattosttrf(a + b, fnumber,20,2);
end;
```

Selezionato Button2, apriamo la scheda Eventi dell'Ispettore Oggetti e selezioniamo l'evento OnClick. Nell'editor facciamo in modo che la procedura abbozzata da Lazarus, inserendo i dati in rosso, diventi la seguente:

```
procedure TForm1.Button2Click(Sender: TObject);
begin
    edit1.Text:='';
    edit2.Text:='';
    label4.Caption:='';
end;
```

Selezionato Button3, apriamo la scheda Eventi dell'Ispettore Oggetti e selezioniamo l'evento OnClick. Nell'editor facciamo in modo che la procedura abbozzata da Lazarus, inserendo i dati in rosso, diventi la seguente:

```
procedure TForm1.Button3Click(Sender: TObject);
begin
    close;
end;
```

Salviamo tutto e compiliamo.

6 Abbellimento della console

Quello che oggi chiamiamo Terminale in Linux e Mac OS X e Prompt dei comandi in Windows, in cui utilizziamo i così detti programmi console, è, in realtà, un emulatore dello schermo dei veri e propri terminali con cui si lavorava con i mainframe e che è poi diventato lo schermo dei primi personal computer, quelli nei quali non c'erano ancora le interfacce grafiche cui siamo abituati ormai da molti anni (X Window System del 1984 entrato in Mac e Linux, Windows del 1985 come ambiente a finestre del sistema operativo MS-DOS, poi diventato lui stesso sistema operativo).

Su quegli schermi si collocavano i caratteri per interfacciarsi con l'utenza su 80 colonne e 24 righe, dimensioni consacrate nel 1971 con il terminale IBM 3270. Successivamente la stessa IBM produsse terminali a 25 righe. Sono praticamente queste le dimensioni degli attuali emulatori di terminale che abbiamo sui nostri computer³.

³A chi interessi conoscere le origini di queste dimensioni posso dire che 80 erano le colonne della prima scheda perforata standard IBM del 1928, che tuttavia aveva solo 12 righe. Forse il numero delle righe del terminale era inizialmente semplicemente il doppio di queste, per rendere il rettangolo di lavoro meno schiacciato e dare allo schermo una forma più da schermo che da scheda perforata.

A quei tempi si trattava di schermi a tubo catodico e ancor oggi la unit del linguaggio Pascal concepita per abbellire lo schermo del terminale su cui si lavorava si chiama `crt`, che è l'acronimo di Cathodic Ray Tube.

Oggi possiamo utilizzare questa unit per abbellire l'emulatore del terminale per il quale era stata concepita.

Per utilizzare la unit dobbiamo importarla con `uses crt;`.

Le procedure che ci mette a disposizione questa unit, prescindendo da quelle ormai anacronistiche⁴, sono le seguenti.

Controllo del display

`textbackground(<colore>)` imposta il colore di fondo;

`<colore>` si indica con

- | | |
|---|------------------------|
| 0 | per il nero |
| 1 | per il blu |
| 2 | per il verde |
| 3 | per il turchese (cyan) |
| 4 | per il rosso |
| 5 | per il magenta |
| 6 | per il marrone |
| 7 | per il grigio chiaro |

`clrscr` ripulisce lo schermo e estende il colore di fondo a tutto lo schermo;

`textcolor(<colore>)` imposta il colore del carattere;

`<colore>` si indica con

- | | | | |
|---|------------------------|----|------------------------|
| 0 | per il nero | 8 | per il grigio chiaro |
| 1 | per il blu | 9 | per il blu chiaro |
| 2 | per il verde | 10 | per il verde chiaro |
| 3 | per il turchese (cyan) | 11 | per il turchese chiaro |
| 4 | per il rosso | 12 | per il rosso chiaro |
| 5 | per il magenta | 13 | per il magenta chiaro |
| 6 | per il marrone | 14 | per il giallo |
| 7 | per il grigio chiaro | 15 | per il bianco |

Controllo del cursore

La posizione del cursore è determinata dalle coordinate `x`, per la posizione sulle colonne, e `y` per la posizione sulle righe. Ricordo che, in eredità dei vecchi terminali, le colonne sono 80 e le righe 25.

`gotoxy(x, y)` posiziona il cursore sulla colonna `x` e sulla riga `y`,

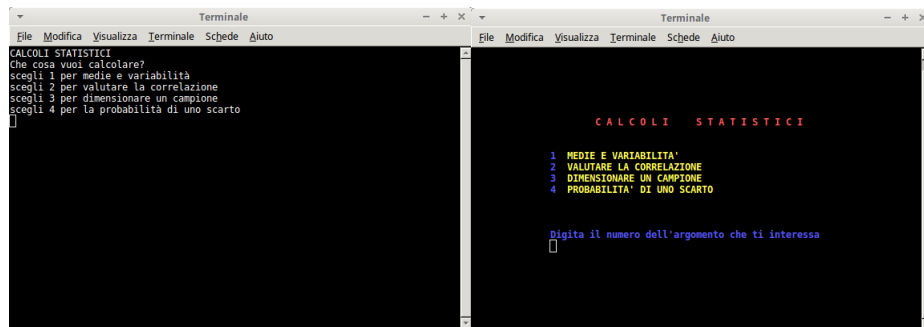
`wherex` ritorna il numero della colonna su cui si trova il cursore,

`wherey` ritorna il numero della riga su cui si trova il cursore.

Può essere quanto serve per rendere un po' più gradevole il freddo emulatore di terminale.

Per evidenziare la differenza tra un terminale non abbellito e un terminale abbellito presento il confronto tra due menu di un ipotetico programma di calcoli statistici, il primo (sulla sinistra) risultato di una procedura scritta con il normale linguaggio, il secondo (sulla destra) risultato di una procedura scritta utilizzando la unit `crt`.

⁴Per esempio, esiste la procedura `window(x1, y1, x2, y2)`, con `x` minore di 80 e `y` minore di 25, per disegnare una finestra più piccola dello schermo intero ai tempi del vero e proprio terminale. Oggi questa procedura disegnerebbe una finestra all'interno dell'emulatore del terminale che, a sua volta, è una finestra minore dello schermo e ciò ritengo non possa interessare più nessuno.



Il codice in linguaggio normale è il seguente:

```
procedure menu;
begin
    writeln('CALCOLI STATISTICI');
    writeln('Che cosa vuoi calcolare?');
    writeln('scegli 1 per medie e variabilità');
    writeln('scegli 2 per valutare la correlazione');
    writeln('scegli 3 per dimensionare un campione');
    writeln('scegli 4 per la probabilità di uno scarto');
end;
```

Il codice utilizzando la unit crt è il seguente:

```
uses crt;
procedure menu;
begin
    textbackground(0);
    clrscr;
    gotoxy(23,7); textcolor(12);
    write('C A L C O L I      S T A T I S T I C I');
    gotoxy(15,10); textcolor(9);
    write('1');
    gotoxy(18,10); textcolor(14);
    write('MEDIE E VARIABILITA''');
    gotoxy(15,11); textcolor(9);
    write('2');
    gotoxy(18,11); textcolor(14);
    write('VALUTARE LA CORRELAZIONE');
    gotoxy(15,12); textcolor(9);
    write('3');
    gotoxy(18,12); textcolor(14);
    write('DIMENSIONARE UN CAMPIONE');
    gotoxy(15,13); textcolor(9);
    write('4');
    gotoxy(18,13); textcolor(14);
    write('PROBABILITA'' DI UNO SCARTO');
    gotoxy(15,17); textcolor(9);
    write('Digita il numero dell''argomento che ti interessa');
    gotoxy(15,18);
end;
```

7 Lavorare con i file

Nella documentazione ufficiale si trovano modalità di trattare file con Free Pascal più sofisticate di quella che presento qui, dove mi propongo semplicemente di descrivere il meccanismo di base.

Variabile di tipo file

La prima cosa da fare è creare una variabile di tipo file su cui lavorare.

Il tipo è quello dei dati che il file contiene o è destinato a contenere.

Nella sezione dichiarativa del programma la sintassi è la seguente:

```
var <nome>: <tipo_file>;
```

dove

<nome> è il nome della variabile: in genere, per brevità, si sceglie `f`,

<tipo_file> può essere espresso con la sintassi

`file of <tipo_dato>` dove <tipo_dato> è uno di quelli visti nel paragrafo 3.1 di questa parte II,

oppure, se si tratta di un comune file di testo, con la parola chiave `textfile`.

Ad esempio:

```
var f: file of char; crea la variabile file f per dati di tipo char,
```

```
var f: file of integer; crea la variabile file f per dati di tipo numero intero,
```

```
var f: textfile; crea la variabile file f per file di testo.
```

Agganciamento della variabile di tipo file a un file sul disco del computer

Nel corpo del programma, con l'istruzione

```
assign(<nome>, <percorso_al_file_su_disco>);
```

assegniamo alla variabile di tipo file (per semplicità `f`), il file su cui lavorare, indicandone il percorso nel file system.

Avendo preventivamente dichiarato, nella sezione dichiarativa

```
var f: textfile;;
```

```
assign(f, '/home/vittorio/Documenti/prova.txt');
```

nel corpo del programma assegna a questa variabile il file `prova.txt` che si trova all'indirizzo indicato nel mio file system Linux. Stesso stile in Mac. In Windows troveremo percorsi del tipo `'C:\.....'`.

E' molto importante che il file indicato contenga dati del tipo assegnato alla variabile file. In proposito rammento che per scrivere testi non va scelto il tipo `char`, a meno che vogliamo memorizzare di volta in volta singoli caratteri: dobbiamo invece scegliere `textfile`, che ci consente di memorizzare parole e frasi.

Creazione del file

Nel corpo del programma, dopo l'istruzione `assign`, con l'istruzione

```
rewrite(<nome>, <percorso_al_file_su_disco>);
```

predisponiamo il file indicato nel percorso alla riscrittura, nel senso che, se il file esiste, viene svuotato per essere riscritto eliminando il contenuto precedente e, se il file non esiste, viene creato vuoto di contenuto.

Aggiornamento del file

Nel corpo del programma, dopo l'istruzione `assign`, con l'istruzione

```
append(<nome>, <percorso_al_file_su_disco>);
```

predisponiamo il file indicato nel percorso ad avere aggiunti altri elementi al suo contenuto.

Lettura del file

Nel corpo del programma, dopo l'istruzione `assign`, con l'istruzione `reset(<nome>, <percorso_al_file_su_disco>);` predisponiamo il file indicato nel percorso ad essere letto.

Azioni sul file

La scrittura sul file predisposto alla scrittura o all'aggiornamento avviene con l'istruzione `writeln(<nome>, <testo_tra_apici>);`

Con l'istruzione

```
writeln(f, 'Sono finalmente riuscito a scrivere su file con Free Pascal');
```

scriviamo la frase indicata nella variabile `f`.

Non vi sono limitazioni al numero dei caratteri che compongono la frase (nonostante la frase sia scritta tra apici non è di tipo `string`).

Affinché la frase raggiunga effettivamente il file agganciato alla variabile `f` occorre far seguire l'istruzione

```
close(f);
```

La lettura dal file predisposto ad essere letto avviene con l'istruzione `readln(<nome>, <variabile>);`

che legge una riga del file fino ad `eof` (end of line).

Per leggere tutto il file occorre instaurare un ciclo fino ad `eof` (end of file), del tipo `repeat readln(<nome>, <variabile>) until eof(<nome>);`

Purtroppo la `<variabile>` in cui viene collocato ciò che si legge deve essere preventivamente dichiarata di tipo `string` e soffre della limitazione di 255 caratteri.

Questa limitazione esclude che Free Pascal possa essere affidabile per leggere file di testo.